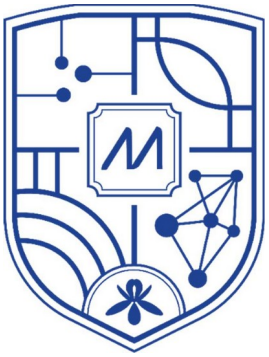
LAPORAN PROYEK
UJIAN TENGAH SEMESTER (UTS)
PEMBELAJARAN MENDALAM

ANALISIS KOMPARATIF MULTI-LAYER PERCEPTRON DAN CONVOLUTIONAL NEURAL NETWORK DENGAN TEKNIK OPTIMASI DAN REGULARISASI PADA KLASIFIKASI PENYAKIT DAUN TOMAT

Comparative Analysis of MLP and CNN with Optimization and Regularization Techniques for Tomato Leaf Disease Classification

Mata Kuliah: Pembelajaran Mendalam
Nama Kelompok: [Nama Kelompok]
Anggota (NIM dan Nama):
[Nama Anggota 1] ([NIM 1])
[Nama Anggota 2] ([NIM 2])
[Nama Anggota 3] ([NIM 3])
[Nama Anggota 4] ([NIM 4])
[Nama Anggota 5] ([NIM 5])



SEMESTER GANJIL
TA. 2026-2027

DAFTAR ISI

DAFTAR ISI....2
DAFTAR GAMBAR....3
DAFTAR TABEL....4
PEMBAGIAN TUGAS....5
BAGIAN A. LAPORAN TEKNIS PROYEK....6
A.1 Kompleksitas Masalah....6
A.1.a Mengapa masalah ini dianggap kompleks?....6
A.1.b Apa tantangan utama dalam menyelesaikan masalah ini?....6
A.2 Dataset yang Digunakan....7
A.2.a Sumber dataset....7
A.2.b Jumlah data, kelas/label, format dataset, dan ukuran data....7
A.2.c Apakah dataset bersifat real-world atau sintetis?....8
A.2.d Seberapa kompleks dataset tersebut?....8
A.3 Exploratory Data Analysis (EDA)....9
A.3.a Distribusi label/kelas....9
A.3.b Word count....9
A.3.c Histogram fitur numerik....9
A.3.d Korelasi antarfitur....10
A.3.e Contoh sampel data per kelas....10
A.3.f Temuan penting dari EDA yang memengaruhi tahap preprocessing....10
A.4 Preprocessing Data....12
A.4.a Pembersihan data (duplikat, data rusak, label salah)....12
A.4.b Normalisasi/standardisasi untuk citra: resize dan rescale piksel....12
A.4.c Augmentasi data....12
A.4.d Penanganan missing value, outlier, dan ketidakseimbangan kelas....13
A.4.e Pembagian data dan pencegahan kebocoran data....13
A.5 Model yang Digunakan....14
A.5.a Jenis model....14
A.5.b Arsitektur model....14
A.5.c Konfigurasi pelatihan....16
A.5.d Alasan pemilihan model dan arsitektur....17
A.5.e Dua fitur unik....17
A.6 Hasil Evaluasi (Akurasi/Error)....21
A.6.a Metrik evaluasi yang digunakan dan alasannya....21
A.6.b Nilai metrik pada data testing....21
A.6.c Grafik loss dan akurasi pada data training dan validation....22
A.6.d Perbandingan antarskenario....23
A.6.e Analisis hasil....24
A.7 Teknologi yang Digunakan....26
A.8 Deployment dan Demo....27
A.8.a Link deployment....27
A.8.b Tahapan deployment langkah demi langkah....27
A.8.c Penjelasan video demo dan deployment....27
A.9 Lampiran....29
A.9.a Cuplikan kode penting....29
A.9.b Grafik evaluasi....34
A.10 Daftar Pustaka....36
BAGIAN B. TAUTAN....37

DAFTAR GAMBAR

Gambar 1. Distribusi kelas pada data mentah (kiri) dan pada tiap split (kanan, skala log).....9
Gambar 2. Histogram intensitas piksel (data latih), kecerahan rata-rata per kelas, dan rata-rata kanal RGB per kelas..... 9
Gambar 3. Contoh enam citra acak per kelas (resolusi 64x64 yang dilihat model).....10
Gambar 4. Perbandingan citra asli 256x256 dengan masukan model 64x64.....11
Gambar 5. Contoh citra asli dan sembilan hasil augmentasi acak.....13
Gambar 6. Diagram arsitektur CNN + regularisasi (model akhir).....14
Gambar 7. Diagram arsitektur MLP + regularisasi (pembanding).....15
Gambar 8. Grad-CAM pada satu contoh benar per kelas (baris atas: citra asli, baris bawah: Grad-CAM)..... 18
Gambar 9. Kurva uji deletion Grad-CAM dibandingkan kontrol (peluang kelas benar dan akurasi).....19
Gambar 10. Selective classification (kiri), distribusi keyakinan model (tengah), dan trade-off ambang (kanan).....20
Gambar 11. Contoh daun non-tomat beserta prediksi dan keputusan terima/tolak.....20
Gambar 12. Confusion matrix CNN + regularisasi pada data uji (angka = jumlah citra).....22
Gambar 13. Kurva loss (atas) dan akurasi (bawah) data training dan validation keempat model; garis titik-titik = epoch dengan val_loss terendah.....22
Gambar 14. Perbandingan accuracy dan macro-F1 antarskenario pada data uji.....23
Gambar 15. Grad-CAM pada prediksi yang salah dengan keyakinan tertinggi (Grad-CAM untuk kelas yang diprediksi).....24
Gambar 16. Confusion matrix keempat model pada data uji.....34
Gambar 17. F1-score per kelas untuk keempat model (data uji).....34
Gambar 18. Grad-CAM pada contoh prediksi yang salah (kelas prediksi).....35
Gambar 19. Contoh daun non-tomat beserta keputusan terima/tolak.....35

DAFTAR TABEL

Tabel 1. Pembagian tugas anggota kelompok.....5
Tabel 2. Rincian kelas, jumlah citra per split, dan bobot kelas.....7
Tabel 3. Ringkasan karakteristik dan kompleksitas dataset.....8
Tabel 4. Jenis dan parameter augmentasi.....12
Tabel 5. Empat model utama yang dibandingkan.....14
Tabel 6. Rincian layer CNN + regularisasi.....14
Tabel 7. Perbandingan jumlah parameter keempat model.....15
Tabel 8. Konfigurasi pelatihan keempat model (seed 42 untuk semua).....16
Tabel 9. Uji deletion: peluang kelas benar (rata-rata) setelah sebagian piksel dihapus.....18
Tabel 10. Uji deletion: akurasi pada citra yang sudah dimasking.....18
Tabel 11. Pengaruh ambang tau pada data uji tomat dan pada daun non-tomat.....19
Tabel 12. Penolakan citra non-tomat per jenis tanaman pada tau terpilih.....20
Tabel 13. Metrik keempat model pada data uji (2.724 citra).....21
Tabel 14. Metrik per kelas untuk CNN + regularisasi pada data uji.....21
Tabel 15. Nilai pada epoch asal bobot yang dievaluasi untuk setiap model.....22
Tabel 16. Perbandingan seluruh skenario yang dijalankan (data uji; satu seed).....23
Tabel 17. Teknologi, framework, dan tools.....26
Tabel 18. Rencana isi video demo.....27
Tabel 19. Hasil model akhir pada contoh aplikasi (tau = 0,945).....28
Tabel 20. Tautan artefak proyek.....37

PEMBAGIAN TUGAS

Tabel 1. Pembagian tugas anggota kelompok

NIM	Nama	Peran	Rincian pekerjaan
[NIM 1]	[Nama Anggota 1]	Data dan EDA	Mengunduh PlantVillage, deduplikasi (MD5 dan perceptual hash), pembagian data berstrata-berkelompok, EDA, pipeline tf.data.
[NIM 2]	[Nama Anggota 2]	Model MLP	Implementasi MLP baseline dan MLP teregulasi, pelatihan, analisis perbandingan dengan CNN.
[NIM 3]	[Nama Anggota 3]	Model CNN dan optimasi	Implementasi CNN baseline dan CNN teregulasi, pengaturan optimizer, LR scheduling, callback, dan skenario ablasi.
[NIM 4]	[Nama Anggota 4]	Fitur unik dan evaluasi	Grad-CAM dan uji deletion, confidence threshold dan uji non-tomat, metrik evaluasi, confusion matrix, analisis kesalahan.
[NIM 5]	[Nama Anggota 5]	Deployment dan dokumentasi	Skrip inferensi, aplikasi Gradio, Hugging Face Spaces, GitHub dan README, video demo, penyusunan laporan.

[Sesuaikan peran dan rincian di atas dengan kontribusi sebenarnya setiap anggota; hapus baris jika anggota kurang dari 5.]

BAGIAN A. LAPORAN TEKNIS PROYEK

A.1 Kompleksitas Masalah

Proyek ini membangun sistem klasifikasi citra daun tomat ke dalam **10 kelas**: sembilan jenis penyakit atau gangguan (bercak bakteri, hawar awal, hawar akhir, jamur daun, bercak septoria, tungau laba-laba, bercak target, virus keriting kuning, dan virus mosaik) serta satu kelas daun sehat. Penyakit daun pada tomat disebabkan oleh bakteri, jamur, virus, dan hama, dan diagnosis visual oleh ahli membutuhkan waktu serta tidak selalu tersedia bagi petani. Klasifikasi otomatis berbasis citra dapat menjadi alat bantu skrining awal [2]. Pada proyek ini, dua keluarga jaringan saraf tiruan dibandingkan secara terkontrol: **Multi-Layer Perceptron (MLP)** yang memperlakukan citra sebagai vektor piksel, dan **Convolutional Neural Network (CNN)** yang memanfaatkan struktur spasial citra. Keduanya dilatih dalam dua versi, yaitu *baseline* dan versi dengan teknik optimasi serta regularisasi dari materi perkuliahan.

A.1.a Mengapa masalah ini dianggap kompleks?

1. **Ketidakseimbangan kelas.** Kelas terbesar (virus keriting kuning, 5.357 citra atau 29,5% dari seluruh data) berjumlah 14,4 kali kelas terkecil (virus mosaik, 373 citra). Model cenderung bias ke kelas mayoritas sehingga akurasi saja dapat menyesatkan; kinerja kelas minoritas harus dipantau lewat precision, recall, dan macro-F1.
2. **Dimensi masukan tinggi.** Satu citra 64x64 RGB memiliki 12.288 nilai piksel. Pada MLP, satu lapisan Dense 256 neuron saja menambah 3.145.984 parameter (12.288 x 256 + 256), sehingga model mudah *overfitting* dan tidak memanfaatkan hubungan antarpiksel yang bertetangga.
3. **Kemiripan visual antarkelas.** Bercak bakteri, bercak target, bercak septoria, dan hawar awal sama-sama tampak sebagai bercak cokelat atau gelap pada daun; perbedaannya terletak pada ukuran, tepi, dan pola bercak. Setelah citra diperkecil ke 64x64, detail halus tersebut menjadi sulit dibedakan.
4. **Variasi kondisi citra.** Posisi dan orientasi daun bebas, pencahayaan dan bayangan berbeda, serta kecerahan rata-rata antarkelas bervariasi dari sekitar 101 hingga 128 (skala 0-255) pada hasil EDA, sehingga model harus kokoh terhadap rotasi, pembalikan, dan perubahan kecerahan.
5. **Risiko kebocoran data.** Ditemukan 14 berkas duplikat persis (MD5) dalam data mentah. Jika duplikat atau citra yang sangat mirip jatuh ke data latih dan uji sekaligus, kinerja uji menjadi terlalu optimistis.
6. **Kesenjangan domain.** Data berupa foto daun tunggal berlatar relatif polos. Pada penggunaan nyata, pengguna dapat mengunggah foto dengan latar kompleks atau bahkan bukan daun tomat; model standar tetap akan memaksakan salah satu dari 10 kelas.

A.1.b Apa tantangan utama dalam menyelesaikan masalah ini?

1. Mendapatkan akurasi tinggi dengan model yang ringan dan waktu pelatihan yang terbatas (eksperimen dijalankan di CPU), sehingga arsitektur harus efisien dalam jumlah parameter dan komputasi.
2. Menangani ketidakseimbangan kelas dan mengevaluasi model secara adil melalui macro-F1 serta analisis per kelas, bukan hanya akurasi total.
3. Mengendalikan *overfitting* dan menstabilkan pelatihan melalui Batch Normalization, Dropout, regularisasi L2, augmentasi data, penjadwalan *learning rate*, dan *early stopping*.

4. Mencegah kebocoran data antara data latih, validasi, dan uji (deduplikasi, pembagian berstrata dan berkelompok, serta pemilihan ambang pada data validasi).
5. Menjelaskan alasan prediksi model agar hasilnya dapat dipercaya pengguna (*explainability*).
6. Menolak masukan yang tidak yakin atau berada di luar domain (bukan daun tomat) agar aplikasi tidak memberikan diagnosis yang menyesatkan.

A.2 Dataset yang Digunakan

A.2.a Sumber dataset

Dataset yang digunakan adalah **PlantVillage** [1], sebuah repositori terbuka citra kesehatan tanaman. Citra diambil dari folder `raw/color` pada repositori GitHub *PlantVillage-Dataset* (https://github.com/spMohanty/PlantVillage-Dataset), dengan hanya mengambil 10 folder kelas tomat (`Tomato___*`). Untuk pengujian fitur penolakan masukan di luar domain (Bab A.5.e), diambil tambahan 600 citra daun non-tomat (kentang, paprika, anggur, dan stroberi sehat; masing-masing 150 citra) dari repositori yang sama. Citra non-tomat ini **tidak pernah dipakai untuk pelatihan**.

A.2.b Jumlah data, kelas/label, format dataset, dan ukuran data

Tabel 2. Rincian kelas, jumlah citra per split, dan bobot kelas

ID	Kelas	Mentah	Setelah dedup	Train	Val	Test	Bobot kelas
0	Bacterial spot (Bercak Bakteri)	2.127	2.127	1.489	318	320	0,853
1	Early blight (Hawar Awal)	1.000	1.000	700	150	150	1,814
2	Late blight (Hawar Akhir)	1.909	1.901	1.331	285	285	0,954
3	Leaf mold (Jamur Daun)	952	952	667	142	143	1,904
4	Septoria leaf spot (Bercak Septoria)	1.771	1.771	1.239	266	266	1,025
5	Spider mites (Tungau Laba-laba)	1.676	1.676	1.172	252	252	1,084
6	Target spot (Bercak Target)	1.404	1.404	983	211	210	1,292
7	Yellow leaf curl virus (Virus Keriting Kuning (YLCV))	5.357	5.357	3.750	804	803	0,339
8	Tomato mosaic virus (Virus Mosaik (ToMV))	373	373	261	57	55	4,866
9	Healthy (Daun Sehat)	1.591	1.585	1.108	237	240	1,146
	Total	**18.160**	**18.146**	**12.700**	**2.722**	**2.724**	

- **Jumlah data:** 18.160 citra mentah; 18.146 citra setelah 14 duplikat persis dibuang.

- **Kelas/label:** 10 kelas (9 penyakit dan 1 sehat), label bertipe bilangan bulat 0-9 (*sparse*).
- **Format:** berkas JPEG berwarna (RGB), resolusi asli 256x256 piksel. Citra diubah menjadi **64x64x3** (12.288 fitur piksel per citra) dan disimpan sebagai larik `uint8` berukuran (18.146, 64, 64, 3).
- **Ukuran split:** 12.700 latih (70%), 2.722 validasi (15%), 2.724 uji (15%).

A.2.c Apakah dataset bersifat real-world atau sintetis?

Dataset bersifat **real-world**: seluruh citra adalah foto daun tomat sungguhan yang diambil pada kondisi terkontrol (satu daun per foto dengan latar relatif polos), bukan citra yang dibangkitkan secara sintetis. Dengan demikian pola penyakit, tekstur, dan variasi pencahayaan merupakan variasi alami, tetapi kondisi pengambilan gambar lebih bersih daripada foto lapangan sehingga terdapat risiko kesenjangan domain saat aplikasi digunakan di kebun (dibahas pada Bab A.6.e).

A.2.d Seberapa kompleks dataset tersebut?

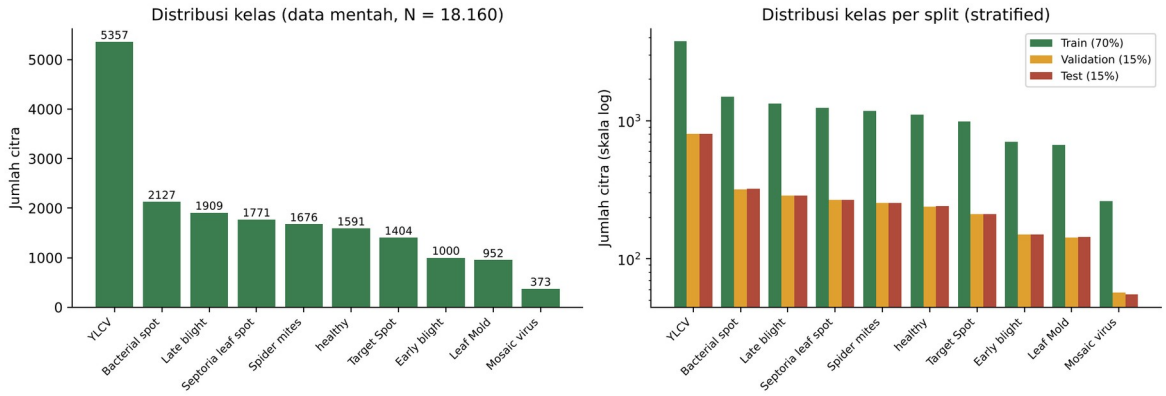
Tabel 3. Ringkasan karakteristik dan kompleksitas dataset

Aspek	Nilai	Implikasi
Ukuran	18.146 citra	Cukup besar untuk melatih CNN kecil dari awal.
Jumlah kelas	10	Klasifikasi multikelas.
Rasio ketidakseimbangan	14,4 : 1	Perlu class weight dan macro-F1.
Dimensi masukan	64 x 64 x 3 = 12.288	MLP memiliki >3 juta parameter; CNN lebih hemat.
Kemiripan antarkelas	Tinggi pada kelompok bercak	Perlu fitur lokal (konvolusi) dan analisis per kelas.
Duplikat	14 persis; 3 pasang mirip	Perlu deduplikasi dan split berkelompok.
Domain	Foto terkontrol, latar polos	Perlu augmentasi dan penolakan masukan di luar domain.

Secara keseluruhan dataset ini berkompleksitas **sedang**: ukurannya memadai dan citranya relatif bersih, tetapi ketidakseimbangan kelas yang cukup besar, kemiripan antarpenyakit, dan risiko kesenjangan domain membuatnya lebih menantang daripada sekadar klasifikasi biner.

A.3 Exploratory Data Analysis (EDA)

A.3.a Distribusi label/kelas



Gambar 1. Distribusi kelas pada data mentah (kiri) dan pada tiap split (kanan, skala log)

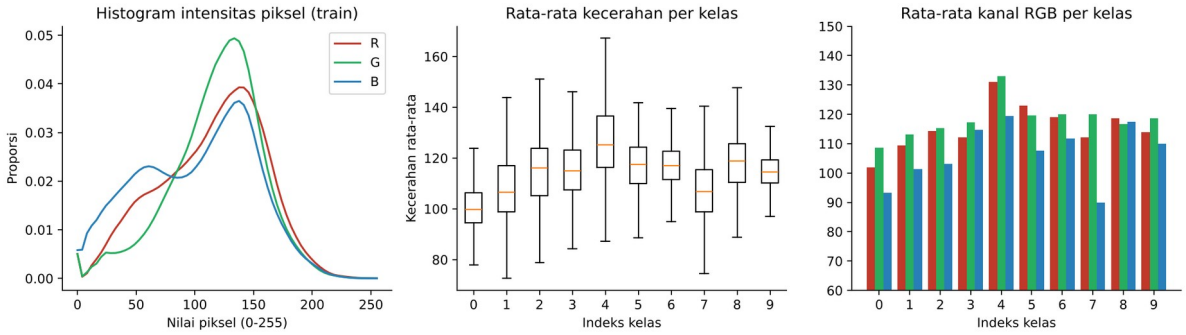
Kelas virus keriting kuning mendominasi (5.357 citra), sedangkan virus mosaik hanya 373 citra. Split berstrata menjaga proporsi kelas yang hampir sama pada data latih, validasi, dan uji; kelas virus mosaik hanya memiliki 57 citra validasi dan 55 citra uji sehingga metriknya lebih berfluktuasi dibanding kelas lain.

A.3.b Word count

Tidak berlaku, karena data berupa citra (bukan teks).

A.3.c Histogram fitur numerik

Fitur tabular tidak tersedia. Sebagai padanannya untuk data citra ditampilkan histogram intensitas piksel per kanal warna, kecerahan rata-rata per kelas, dan rata-rata kanal RGB per kelas.



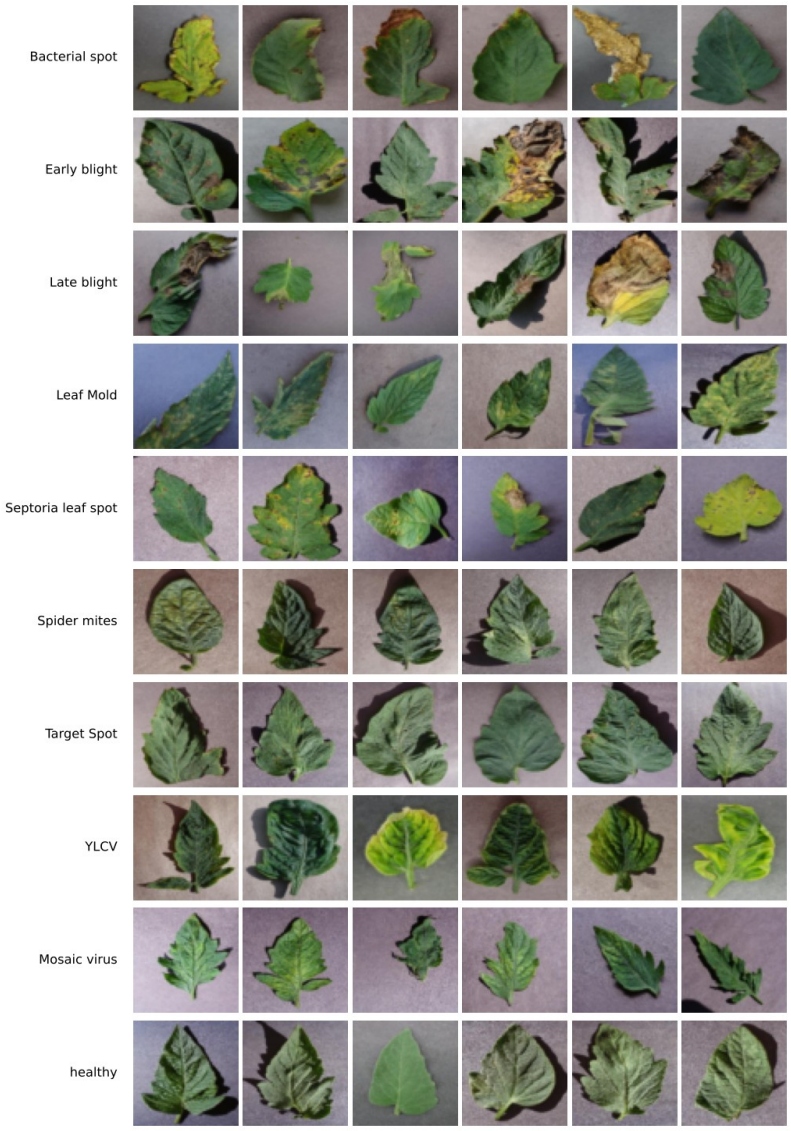
Gambar 2. Histogram intensitas piksel (data latih), kecerahan rata-rata per kelas, dan rata-rata kanal RGB per kelas

Intensitas piksel menempati seluruh rentang 0-255 sehingga cukup diskalakan ke [0, 1] dengan pembagian 255. Rata-rata kanal pada data latih adalah R = 0,450, G = 0,466, B = 0,401 (simpangan baku 0,170, 0,147, 0,187); kanal hijau dominan, wajar untuk citra daun. Kecerahan dan komposisi warna berbeda antarkelas, misalnya kelas virus keriting kuning memiliki kanal biru paling rendah (daun menguning), tetapi perbedaan latar dan pencahayaan antarkelas juga ikut menyumbang, sehingga model berisiko mempelajari warna latar sebagai jalan pintas.

A.3.d Korelasi antarfitur

Tidak berlaku untuk data citra: fitur adalah piksel mentah (12.288 dimensi) yang berkorelasi spasial kuat dengan piksel tetangganya. Korelasi spasial inilah yang dimanfaatkan oleh operasi konvolusi pada CNN dan diabaikan oleh MLP yang meratakan (*flatten*) citra.

A.3.e Contoh sampel data per kelas



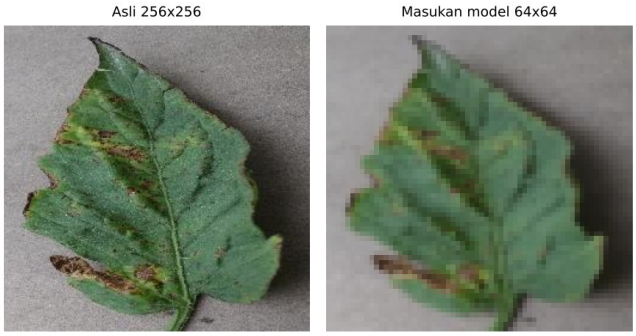
Gambar 3. Contoh enam citra acak per kelas (resolusi 64x64 yang dilihat model)

Pengamatan visual menemukan citra dengan **latar hitam** (piksel tepi hampir hitam, rata-rata kecerahan cincin 4 piksel terluar < 30). Sebanyak 146 citra (0,8% dari seluruh data) berlatar hitam dan **seluruhnya berasal dari kelas Late blight** (146 dari 1.901 citra kelas tersebut, 7,7%); kelas lain tidak memiliki satu pun. Ini adalah artefak dataset: model berpotensi mempelajari "latar hitam berarti Late blight" sebagai jalan pintas. Dampaknya diperiksa pada Bab A.6.e.

A.3.f Temuan penting dari EDA yang memengaruhi tahap preprocessing

1. Ketidakseimbangan kelas 14,4:1 → pembagian data berstrata, **class weight** pada fungsi loss, dan pelaporan **macro-F1**.

2. 14 duplikat persis → **dibuang** sebelum pembagian data; pengecekan near-duplicate dengan *perceptual hash* → split **berkelompok**.
3. Citra seragam 256x256 dan dapat diperkecil tanpa kehilangan pola penyakit yang utama → **resize 64x64** agar pelatihan di CPU layak.
4. Intensitas piksel 0-255 → **rescale** ke [0, 1].
5. Kecerahan dan warna berbeda antarkelas serta orientasi daun bebas → **augmentasi** (flip, rotasi, zoom, brightness) agar model tidak bergantung pada latar atau posisi.
6. Foto berlatar polos → perlu mekanisme **penolakan masukan** di luar domain pada aplikasi.
7. Latar hitam eksklusif pada Late blight → tidak dibuang (agar data tidak berkurang), tetapi kinerja pada subset ini dilaporkan terpisah dan Grad-CAM dipakai untuk memeriksa ketergantungan pada latar.



Gambar 4. Perbandingan citra asli 256x256 dengan masukan model 64x64

A.4 Preprocessing Data

A.4.a Pembersihan data (duplikat, data rusak, label salah)

1. **Duplikat persis.** Setiap berkas di-hash dengan MD5; 14 berkas identik (di dalam kelas yang sama) dibuang sehingga tersisa 18.146 citra. Alasan: duplikat yang terbagi ke latih dan uji menggelembungkan metrik uji.
2. **Near-duplicate.** Setiap citra 64x64 diubah menjadi *difference hash* 256-bit; pasangan dengan jarak Hamming ≤ 12 dikelompokkan (*union-find*). Ditemukan 3 pasangan (1 kelompok) dan tidak ada kelompok lintas kelas. Percobaan awal dengan hash 64-bit menghasilkan banyak kelompok palsu (daun berbeda yang tampak mirip pada resolusi rendah, diperiksa secara visual), sehingga diganti dengan hash 256-bit yang lebih diskriminatif.
3. **Data rusak.** Seluruh citra dapat dibuka dan dikonversi ke RGB tanpa galat; tidak ada citra yang dibuang karena rusak.
4. **Label salah.** Tidak ada duplikat yang berbeda label (tidak ada indikasi *label noise* lintas kelas dari pemeriksaan duplikat). Label berasal dari nama folder PlantVillage dan tidak diubah.

A.4.b Normalisasi/standardisasi untuk citra: resize dan rescale piksel

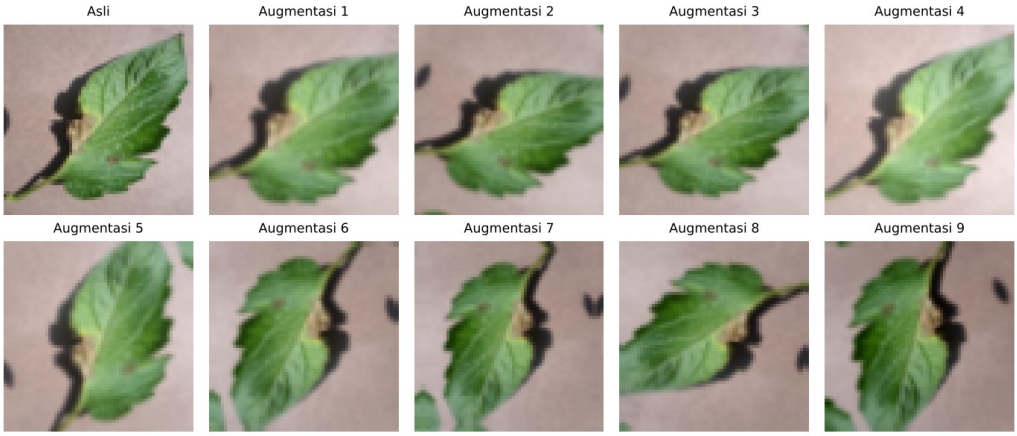
- **Resize** 256x256 → 64x64 dengan interpolasi bilinear (PIL, dengan *antialiasing* saat pengecilan). Alasan: menurunkan biaya komputasi dan memori hingga 16 kali sehingga seluruh skenario dapat dilatih di CPU; pola bercak utama masih terlihat.
- **Rescale** nilai piksel ke [0, 1] (dibagi 255) di dalam *input pipeline* `tf.data`. Standardisasi per kanal tidak dipakai; stabilitas pelatihan dijaga oleh Batch Normalization pada model teregulasi. Skala yang sama diterapkan pada saat inferensi di aplikasi.
- **Input pipeline** `tf.data`: `from_tensor_slices` → `shuffle` (seluruh data latih, diacak ulang tiap epoch) → `map` (cast + rescale) → `batch(64)` → `prefetch(AUTOTUNE)` sesuai materi pertemuan 2.

A.4.c Augmentasi data

Augmentasi diterapkan sebagai lapisan Keras di awal model teregulasi, sehingga hanya aktif saat pelatihan dan otomatis nonaktif saat validasi, pengujian, dan inferensi. Augmentasi dilakukan **setelah** pembagian data sehingga tidak ada citra turunan dari data latih yang masuk ke validasi atau uji.

Tabel 4. Jenis dan parameter augmentasi

Layer	Parameter	Tujuan
RandomFlip	"horizontal_and_vertical"	Daun dapat berorientasi apa pun.
RandomRotation	factor = 0,08 (sekitar +/- 29 derajat)	Variasi sudut pengambilan foto.
RandomZoom	factor = 0,1	Variasi jarak dan ukuran daun.
RandomBrightness	factor = 0,15; value_range = (0, 1)	Variasi pencahayaan.



Gambar 5. Contoh citra asli dan sembilan hasil augmentasi acak

A.4.d Penanganan missing value, outlier, dan ketidakseimbangan kelas

- **Missing value dan outlier:** tidak ada (tidak ada nilai piksel kosong dan semua citra berukuran sama); tidak ada langkah khusus yang diperlukan.
- **Ketidakseimbangan kelas:** digunakan **class weight** berimbang yang dihitung hanya dari data latih, w_c = N / (K x n_c) dengan N = 12.700, K = 10, dan n_c jumlah citra latih kelas c. Bobot berkisar dari 0,339 (virus keriting kuning) hingga 4,866 (virus mosaik), lihat tabel rincian kelas pada Bab A.2.b. Oversampling tidak dipakai agar tidak menggandakan citra yang sama. Evaluasi memakai macro-F1 untuk menilai semua kelas secara setara. Dampaknya pada kelas minoritas dilihat dari F1 per kelas (Bab A.6.b dan A.6.e).

A.4.e Pembagian data dan pencegahan kebocoran data

Data dibagi **70% latih (12.700), 15% validasi (2.722), dan 15% uji (2.724)** dengan `StratifiedGroupKFold` (seed 42): proporsi kelas dijaga (*stratified*) dan citra pada kelompok near-duplicate yang sama selalu berada di split yang sama (*group*). Data validasi dipakai untuk *early stopping*, penjadwalan *learning rate*, dan pemilihan ambang keyakinan; data uji hanya dipakai sekali di akhir untuk pelaporan.

- Deduplikasi dilakukan **sebelum** pembagian data.
- Augmentasi dan class weight dihitung/diterapkan hanya pada data latih; rescale bersifat tetap (dibagi 255) sehingga tidak ada statistik yang “bocor” dari validasi/uji.
- Ambang keyakinan (tau) dipilih pada data validasi, bukan data uji.
- Seluruh skenario memakai split, seed, dan pipeline yang identik sehingga perbandingan antarmodel adil.

A.5 Model yang Digunakan

A.5.a Jenis model

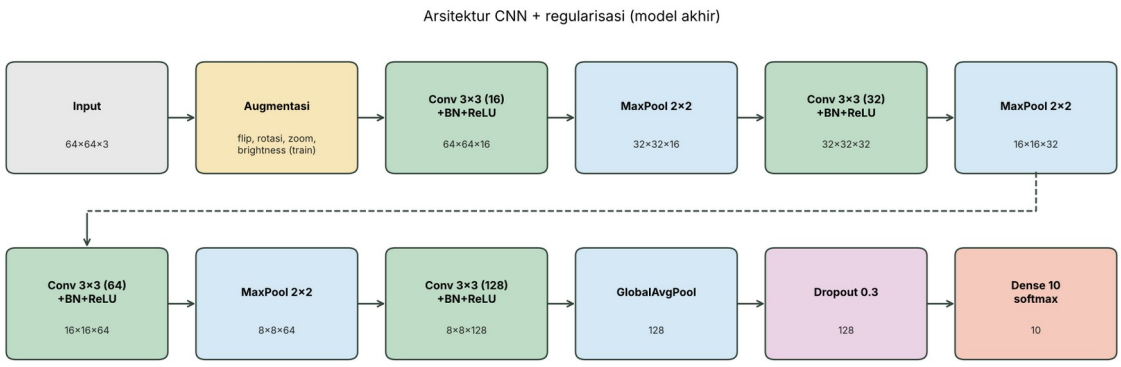
Sesuai batasan proyek, hanya dua keluarga model yang digunakan, keduanya dilatih **dari awal** (tanpa *transfer learning*): **Deep Neural Network berbentuk MLP** dan **CNN**. Masing-masing dibuat dalam dua versi sehingga terbentuk empat model utama:

Tabel 5. Empat model utama yang dibandingkan

Model	Keluarga	Teknik optimasi dan regularisasi	Peran
MLP baseline	MLP	Tidak ada (Adam, lr konstan, EarlyStopping saja)	Pembanding dasar untuk MLP
MLP + regularisasi	MLP	BN, Dropout, L2, augmentasi, class weight, cosine annealing	Mengukur manfaat resep regularisasi pada MLP
CNN baseline	CNN	Tidak ada (Adam, lr konstan, EarlyStopping saja)	CNN dasar sesuai materi pertemuan 6
CNN + regularisasi	CNN	BN, Dropout, L2, augmentasi, class weight, cosine annealing	Model akhir yang di-deploy

Transfer learning (mis. MobileNet) tidak dipakai karena fokus judul proyek adalah perbandingan MLP dan CNN beserta teknik optimasi dan regularisasinya, bukan perbandingan dengan model terlatih berukuran besar; selain itu seluruh eksperimen dijalankan di CPU.

A.5.b Arsitektur model



Gambar 6. Diagram arsitektur CNN + regularisasi (model akhir)

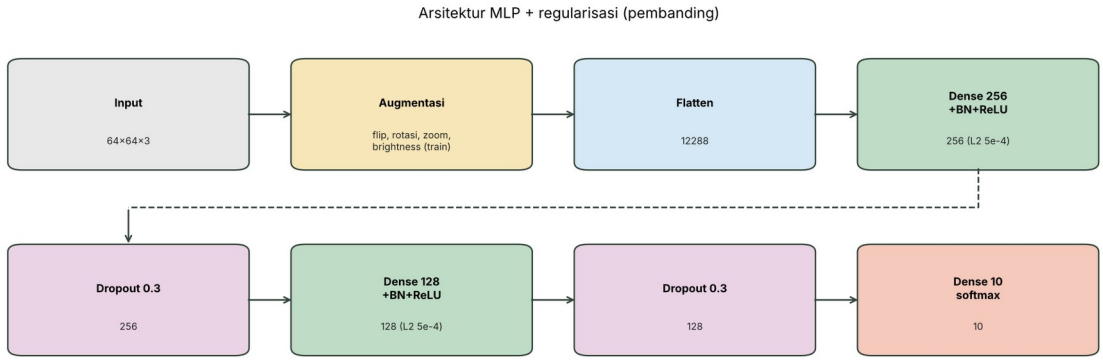
Model akhir terdiri atas empat blok konvolusi dengan jumlah filter yang meningkat (16, 32, 64, 128). Tiga blok pertama diikuti *max pooling* 2x2 yang menurunkan resolusi dari 64x64 menjadi 8x8. Setiap konvolusi berukuran 3x3 dengan *padding* `same` dan tanpa bias karena langsung diikuti Batch Normalization, kemudian aktivasi ReLU. Kepala klasifikasi memakai *Global Average Pooling* (menggantikan *Flatten* + Dense besar), Dropout 0,3, dan lapisan Dense 10 neuron dengan aktivasi softmax. Total parameter **99.450** (98.970 terlatih dan 480 statistik BN yang tidak dilatih).

Tabel 6. Rincian layer CNN + regularisasi

Layer	Tipe	Ukuran output	Parameter
augmentasi	Sequential	(64, 64, 3)	0

Layer	Tipe	Ukuran output	Parameter
conv1	Conv2D	(64, 64, 16)	432
bn1	BatchNormalization	(64, 64, 16)	64
relu1	Activation	(64, 64, 16)	0
pool1	MaxPooling2D	(32, 32, 16)	0
conv2	Conv2D	(32, 32, 32)	4.608
bn2	BatchNormalization	(32, 32, 32)	128
relu2	Activation	(32, 32, 32)	0
pool2	MaxPooling2D	(16, 16, 32)	0
conv3	Conv2D	(16, 16, 64)	18.432
bn3	BatchNormalization	(16, 16, 64)	256
relu3	Activation	(16, 16, 64)	0
pool3	MaxPooling2D	(8, 8, 64)	0
conv4	Conv2D	(8, 8, 128)	73.728
bn4	BatchNormalization	(8, 8, 128)	512
relu4	Activation	(8, 8, 128)	0
gap	GlobalAveragePooling2D	(128,)	0
dropout	Dropout	(128,)	0
logits	Dense	(10,)	1.290
Total			**99.450**

Contoh perhitungan: `conv1` memiliki 3 x 3 x 3 x 16 = 432 parameter (tanpa bias); `conv2` memiliki 3 x 3 x 16 x 32 = 4.608; `conv4` memiliki 3 x 3 x 64 x 128 = 73.728; Batch Normalization menyimpan 4 vektor per kanal (gamma, beta, rata-rata, varians), misalnya 4 x 16 = 64 pada `bn1`; Dense `logits` memiliki 128 x 10 + 10 = 1.290.



Gambar 7. Diagram arsitektur MLP + regularisasi (pembanding)

Tabel 7. Perbandingan jumlah parameter keempat model

Model	Struktur	Total parameter	Terlatih
MLP baseline	Flatten - Dense 256 - Dense 128 - Dense 10	3.180.170	3.180.170

Model	Struktur	Total parameter	Terlatih
MLP + regularisasi	Flatten - 2 x (Dense, BN, ReLU, Dropout 0,3) - Dense 10	3.181.322	3.180.554
CNN baseline	3 x (Conv 3x3, MaxPool) - Flatten - Dense 128 - Dense 10	684.490	684.490
CNN + regularisasi	4 x (Conv, BN, ReLU) + 3 MaxPool - GAP - Dropout - Dense 10	99.450	98.970

CNN + regularisasi hanya memiliki 32 kali lebih sedikit parameter dibanding MLP dan sekitar 6,9 kali lebih sedikit dibanding CNN baseline; penghematan terutama berasal dari penggantian *Flatten* + Dense 128 (589.952 parameter pada CNN baseline) dengan *Global Average Pooling*.

A.5.c Konfigurasi pelatihan

Tabel 8. Konfigurasi pelatihan keempat model (seed 42 untuk semua)

Parameter	Baseline (MLP & CNN)	Teregulasi (MLP & CNN)
Optimizer	Adam (lr = 1e-3)	Adam (lr = 1e-3)
Learning rate schedule	Konstan	Cosine annealing per epoch: 1e-3 menurun ke 1e-5 selama 18 epoch
Batch size	64	64
Epoch maksimum	30	18 (sesuai panjang jadwal cosine)
Fungsi loss	Sparse categorical crossentropy	Sparse categorical crossentropy
Early stopping	val_loss, patience 6, restore_best_weights	val_loss, patience 6, restore_best_weights
Batch Normalization	Tidak	Ya (momentum 0,9)
Dropout	Tidak	0,3
Regularisasi L2	Tidak	1e-4 pada kernel Conv/Dense
Augmentasi	Tidak	Flip, rotasi, zoom, brightness
Class weight	Tidak	Ya (berimbang)
Backup pelatihan	BackupAndRestore (per epoch)	BackupAndRestore (per epoch)

Catatan iterasi resep regularisasi. Resep CNN teregulasi disempurnakan melalui tiga iterasi berdasarkan perilaku data validasi (bukan data uji). Iterasi 1 (momentum BN 0,99, L2 5e-4, ReduceLROnPlateau): akurasi validasi berosilasi lebar pada 18 epoch pertama (0,18 sampai 0,92) sebelum sesi komputasi terputus. Iterasi 2 (momentum BN diturunkan ke 0,9): masih berosilasi (0,34 sampai 0,92 pada 17 epoch) dan sesi kembali terputus, sehingga momentum bukan satu-satunya penyebab. Iterasi 3 (final): L2 diturunkan ke 1e-4 dan *learning rate* konstan/ReduceLROnPlateau diganti cosine annealing 18 epoch, dengan dugaan bahwa L2 yang kuat pada kernel yang diikuti Batch Normalization dan *learning rate* 1e-3 yang tinggi menyebabkan ketidakstabilan. Karena ketiga perubahan dilakukan bersamaan, kontribusi masing-masing tidak dapat dipisahkan. Log iterasi 1 dan 2 diarsipkan pada folder `results/archive`. Pelatihan juga dilengkapi *backup* per epoch sehingga dapat dilanjutkan jika sesi komputasi terputus.

A.5.d Alasan pemilihan model dan arsitektur

1. **MLP sebagai pembanding.** MLP adalah arsitektur paling sederhana (materi pertemuan 3) dan memperlihatkan seberapa jauh citra dapat diklasifikasikan tanpa memanfaatkan struktur spasial. Membandingkannya dengan CNN pada data dan pipeline yang identik menjawab pertanyaan utama judul proyek.
2. **CNN untuk citra.** Konvolusi menggunakan koneksi lokal dan berbagi bobot, sehingga mendeteksi bercak, tepi, dan tekstur di mana pun posisinya dengan parameter jauh lebih sedikit (materi pertemuan 6-7). Ini sesuai untuk pola penyakit yang bersifat lokal.
3. **Filter bertingkat 16-32-64-128 dan pooling.** Fitur sederhana (tepi, warna) dipelajari di lapisan awal pada resolusi tinggi, fitur yang lebih abstrak di lapisan dalam. Lebar lapisan awal sengaja kecil (16) karena biaya komputasi terbesar berada pada resolusi 64x64; percobaan awal dengan 32 filter di lapisan pertama berjalan sekitar 4 kali lebih lambat per epoch di CPU yang digunakan.
4. **Batch Normalization sebelum ReLU** (Dense/Conv tanpa bias) menstabilkan dan mempercepat pelatihan serta memberi efek regularisasi ringan [3] (materi pertemuan 4).
5. **Dropout 0,3 dan regularisasi L2** menekan *co-adaptation* neuron dan memperkecil bobot untuk mengurangi *overfitting* [4]; **augmentasi** memperbanyak variasi data secara efektif tanpa menambah data.
6. **Global Average Pooling** mengurangi parameter dan risiko *overfitting* dibanding *Flatten* + Dense besar.
7. **Adam + cosine annealing + EarlyStopping.** Adam adaptif dan umumnya konvergen cepat [5]; penurunan *learning rate* secara mulus (cosine) membantu penyempurnaan akhir dan menstabilkan validasi pada epoch-epoch akhir; *early stopping* dengan *restore_best_weights* menghentikan pelatihan ketika validasi tidak membaik (materi pertemuan 5).
8. **Class weight** mengimbangi ketidakseimbangan kelas pada fungsi loss tanpa menggandakan data.

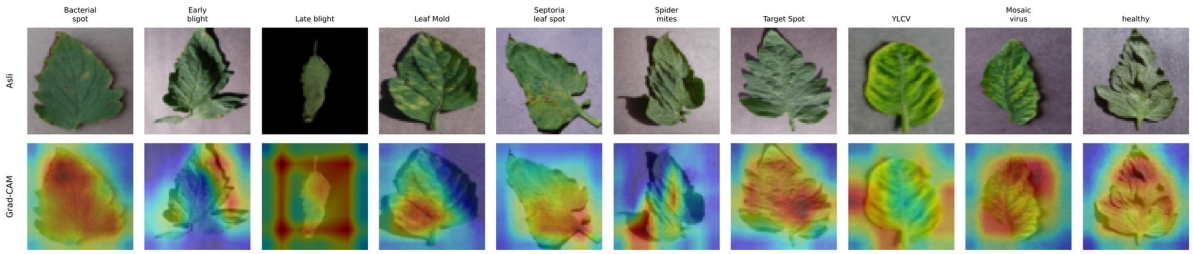
Bukti eksperimen (perbandingan dengan *baseline* pada data yang sama, termasuk hasil yang tidak sesuai harapan) disajikan secara apa adanya pada Bab A.6.

A.5.e Dua fitur unik

Dua fitur tambahan dirancang untuk menjawab dua kelemahan nyata pada klasifikasi citra penyakit tanaman: model sulit dipercaya karena tidak menjelaskan alasannya, dan model selalu memaksakan satu dari 10 kelas meskipun masukannya bukan daun tomat.

Fitur unik 1: Grad-CAM (penjelasan visual prediksi)

- **Apa:** peta panas yang menunjukkan bagian citra yang paling memengaruhi skor kelas prediksi (Selvaraju dkk., 2017) [6].
- **Mengapa dibutuhkan:** pengguna dapat memeriksa apakah model "melihat" bercak pada daun atau justru latar belakang; ini penting untuk kepercayaan dan untuk menganalisis kesalahan. Pada dataset ini terdapat artefak latar hitam yang hanya ada pada kelas Late blight (Bab A.3.e), sehingga pemeriksaan semacam ini relevan.
- **Implementasi:** Grad-CAM dihitung pada keluaran ReLU blok konvolusi terakhir (`relu4`, 8x8x128). Gradien skor logit kelas terhadap peta fitur dirata-ratakan per kanal menjadi bobot alpha; peta = ReLU(jumlah alpha x peta fitur), dinormalisasi ke [0, 1], lalu di-*upsample* bilinear ke 64x64 dan ditumpuk pada citra (colormap jet). Karena kepala model berupa GAP + Dense, hasilnya ekuivalen dengan CAM. Kode: kelas `Predictor.gradcam` (Lampiran, Kode 5a); ditampilkan pada aplikasi.



Gambar 8. Grad-CAM pada satu contoh benar per kelas (baris atas: citra asli, baris bawah: Grad-CAM)

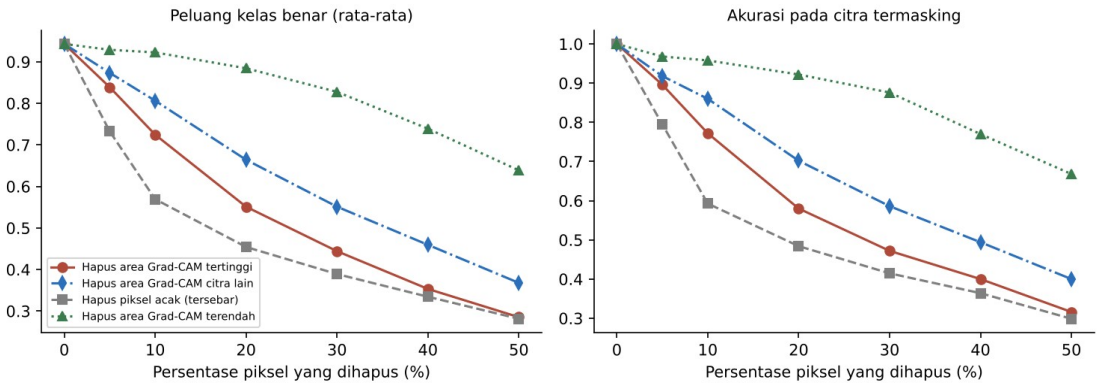
Bukti dampak (kuantitatif, uji deletion). Untuk menguji apakah peta Grad-CAM benar-benar menunjukkan area yang dipakai model (*faithfulness*), 500 citra uji yang diklasifikasikan benar diambil acak. Sebagian piksel dihapus (diganti warna rata-rata citra itu) menurut empat urutan: (a) piksel dengan Grad-CAM **tertinggi** lebih dulu; (b) piksel dengan Grad-CAM tertinggi dari **citra lain** (area menyatu tanpa informasi tentang citra ini, sebagai kontrol); (c) piksel **acak** yang tersebar; dan (d) piksel dengan Grad-CAM **terendah** lebih dulu. Jika Grad-CAM informatif, urutan (a) harus menurunkan peluang kelas benar paling cepat dan urutan (d) paling lambat. Urutan (c) menambahkan derau tersebar sehingga bukan pembanding yang adil bagi area yang menyatu; kontrol (b) lebih tepat.

Tabel 9. Uji deletion: peluang kelas benar (rata-rata) setelah sebagian piksel dihapus

Piksel dihapus	(a) Grad-CAM tertinggi	(b) Grad-CAM citra lain	(c) Piksel acak	(d) Grad-CAM terendah
0%	0,943	0,943	0,943	0,943
5%	0,839	0,873	0,733	0,929
10%	0,725	0,806	0,569	0,923
20%	0,551	0,664	0,454	0,884
30%	0,444	0,551	0,389	0,828
40%	0,353	0,459	0,334	0,739
50%	0,285	0,368	0,281	0,639

Tabel 10. Uji deletion: akurasi pada citra yang sudah dimasking

Piksel dihapus	(a) Grad-CAM tertinggi	(b) Grad-CAM citra lain	(c) Piksel acak	(d) Grad-CAM terendah
0%	100,0%	100,0%	100,0%	100,0%
5%	89,6%	91,8%	79,5%	96,8%
10%	77,2%	86,0%	59,3%	95,8%
20%	58,0%	70,2%	48,4%	92,2%
30%	47,2%	58,6%	41,5%	87,6%
40%	40,0%	49,4%	36,4%	77,0%
50%	31,6%	40,0%	29,9%	66,8%



Gambar 9. Kurva uji deletion Grad-CAM dibandingkan kontrol (peluang kelas benar dan akurasi)

Pada penghapusan 20% piksel, peluang kelas benar turun sebesar 0,392 untuk (a), 0,279 untuk (b), 0,489 untuk (c), dan 0,058 untuk (d); akurasi menjadi 58,0%, 70,2%, 48,4%, dan 92,2%. Menghapus area Grad-CAM tertinggi jauh lebih merusak daripada menghapus area terendah, sehingga peta mampu membedakan bagian citra yang penting dan tidak penting bagi prediksi. Peta spesifik-citra juga lebih merusak dibanding area yang diambil dari citra lain, sehingga informasinya lebih dari sekadar posisi daun yang umumnya berada di tengah citra. Penghapusan piksel acak yang tersebar justru lebih merusak daripada area Grad-CAM tertinggi karena derau tersebar menggeser distribusi masukan lebih kuat daripada satu area yang menyatu; karena itu urutan (c) tidak dijadikan dasar kesimpulan.

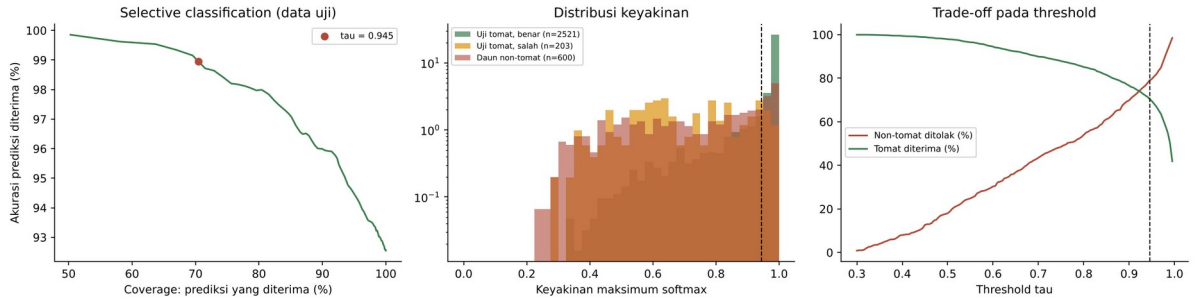
Fitur unik 2: confidence threshold untuk menolak prediksi tidak yakin dan masukan di luar domain

- **Apa:** model hanya memberi keputusan jika probabilitas softmax maksimum (*maximum softmax probability*, MSP) [7] mencapai ambang tau; jika tidak, prediksi **ditolak** dan pengguna diminta memotret ulang (*selective classification*) [8].
- **Mengapa dibutuhkan:** model softmax standar selalu memilih salah satu dari 10 kelas, termasuk untuk foto yang bukan daun tomat. Pada aplikasi nyata ini berbahaya karena menghasilkan diagnosis penyakit yang tidak berdasar.
- **Implementasi:** tau dipilih **pada data validasi** sebagai nilai terkecil pada grid dengan langkah 0,005 sehingga akurasi prediksi yang diterima pada data validasi mencapai minimal 0,99, menghasilkan **tau = 0,945**. Nilai ini disimpan di `model/config.json` dan dapat diubah pengguna lewat slider pada aplikasi. Kode: `Predictor.predict` (Lampiran, Kode 5a) dan pemilihan tau (Kode 5b).
- **Skenario uji:** (i) data uji tomat (2.724 citra) untuk mengukur *coverage* dan akurasi prediksi yang diterima; (ii) 600 citra daun non-tomat (kentang, paprika, anggur, stroberi; tidak pernah dilatih) sebagai masukan di luar domain.

Tabel 11. Pengaruh ambang tau pada data uji tomat dan pada daun non-tomat

Ambang tau	Coverage (tomat diterima)	Akurasi prediksi diterima	Kesalahan yang lolos	Tomat ditolak	Non-tomat ditolak
0 (tanpa fitur)	100,0%	92,55%	203	0,0%	0,0%
0,50	97,9%	93,48%	174	2,1%	18,0%
0,70	89,9%	96,00%	98	10,1%	43,5%
0,80	85,1%	97,07%	68	14,9%	54,0%

Ambang tau	Coverage (tomat diterima)	Akurasi prediksi diterima	Kesalahan yang lolos	Tomat ditolak	Non-tomat ditolak
0,90	76,6%	98,18%	38	23,4%	69,8%
0,95	69,5%	99,16%	16	30,5%	79,7%
0,99	50,3%	99,85%	2	49,7%	95,8%
0,945 (terpilih)	70,5%	98,96%	20	29,5%	79,0%

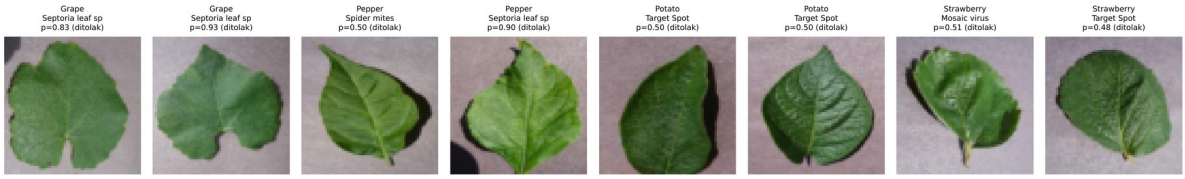


Gambar 10. Selective classification (kiri), distribusi keyakinan model (tengah), dan trade-off ambang (kanan)

Bukti dampak. Tanpa fitur ini (tau = 0), akurasi uji tomat adalah 92,55% dengan 203 kesalahan, dan **0%** citra non-tomat ditolak (seluruh 600 citra tetap dipaksa masuk ke salah satu kelas penyakit tomat). Dengan tau = 0,945, 70,5% citra tomat tetap diterima dengan akurasi 98,96% (kesalahan yang lolos turun dari 203 menjadi 20), dan 79,0% citra non-tomat ditolak. Kemampuan membedakan citra tomat dan non-tomat berdasarkan MSP memiliki AUROC 0,828; bila ambang diatur agar 95% citra tomat diterima, 71,0% citra non-tomat masih lolos (FPR@95TPR).

Tabel 12. Penolakan citra non-tomat per jenis tanaman pada tau terpilih

Tanaman non-tomat	Citra	Ditolak	Kelas tomat yang paling sering diberikan
Grape	150	82,7%	Septoria leaf spot
Pepper (bell)	150	70,7%	Septoria leaf spot
Potato	150	78,0%	Septoria leaf spot
Strawberry	150	84,7%	Septoria leaf spot



Gambar 11. Contoh daun non-tomat beserta prediksi dan keputusan terima/tolak

Fitur ini bukan pendeteksi OOD yang sempurna: MSP dapat tetap tinggi pada sebagian masukan non-tomat, terutama daun yang mirip secara visual dan berlatar sama dengan data latih, sehingga sebagian citra non-tomat tetap lolos. Kalibrasi keyakinan (ECE, 15 *bin*) pada data uji adalah 0,0138 untuk CNN + regularisasi, dibandingkan 0,0265 (MLP baseline) dan 0,0170 (CNN baseline). Ambang tau juga menimbulkan kompromi: semakin tinggi tau, semakin banyak citra tomat valid yang ikut ditolak (kolom “Tomat ditolak”).

A.6 Hasil Evaluasi (Akurasi/Error)

A.6.a Metrik evaluasi yang digunakan dan alasannya

- **Accuracy:** proporsi prediksi benar; mudah dipahami tetapi dapat menyesatkan pada data tidak seimbang.
- **Precision, recall, dan F1-score per kelas** serta rata-rata **macro** (setiap kelas berbobot sama) dan **weighted** (berbobot jumlah data). **Macro-F1 menjadi metrik utama** karena kelas minoritas (mis. virus mosaik) sama pentingnya dengan kelas mayoritas.
- **Recall** penting karena kesalahan jenis *false negative* (penyakit tidak terdeteksi) lebih merugikan petani daripada *false positive*.
- **Confusion matrix** untuk melihat pasangan kelas yang tertukar.
- **Selective accuracy/coverage dan ECE** untuk mengevaluasi fitur confidence threshold dan kalibrasi keyakinan.
- **Loss dan accuracy training/validation per epoch** untuk mendeteksi *overfitting* atau *underfitting*.

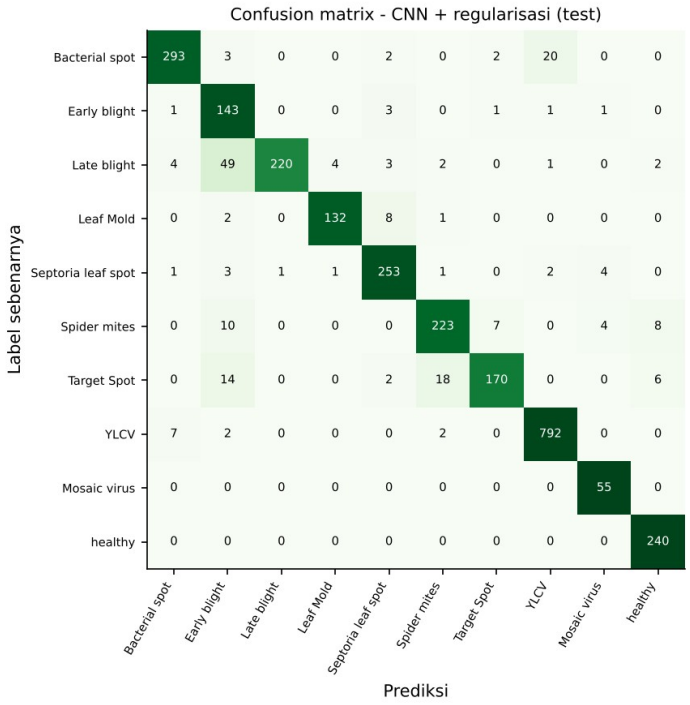
A.6.b Nilai metrik pada data testing

Tabel 13. Metrik keempat model pada data uji (2.724 citra)

Model	Accuracy	Macro precision	Macro recall	Macro F1	Weighted F1
MLP baseline	0,8139	0,7911	0,7563	0,7682	0,8104
MLP + regularisasi	0,8249	0,7720	0,8147	0,7844	0,8256
CNN baseline	0,9460	0,9331	0,9272	0,9299	0,9458
CNN + regularisasi	0,9255	0,9098	0,9196	0,9088	0,9266

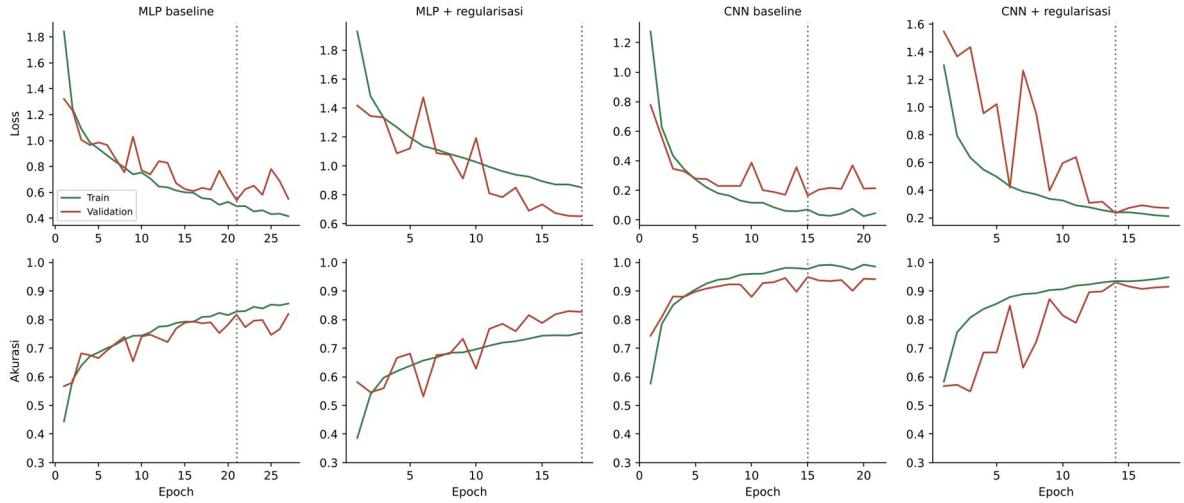
Tabel 14. Metrik per kelas untuk CNN + regularisasi pada data uji

Kelas	Precision	Recall	F1-score	Jumlah uji
Bacterial spot	0,958	0,916	0,936	320
Early blight	0,633	0,953	0,761	150
Late blight	0,996	0,772	0,870	285
Leaf Mold	0,964	0,923	0,943	143
Septoria leaf spot	0,934	0,951	0,942	266
Spider mites	0,903	0,885	0,894	252
Target Spot	0,944	0,809	0,872	210
YLCV	0,971	0,986	0,978	803
Mosaic virus	0,859	1,000	0,924	55
healthy	0,938	1,000	0,968	240
Macro avg	**0,910**	**0,920**	**0,909**	**2724**



Gambar 12. Confusion matrix CNN + regularisasi pada data uji (angka = jumlah citra)

A.6.c Grafik loss dan akurasi pada data training dan validation



Gambar 13. Kurva loss (atas) dan akurasi (bawah) data training dan validation keempat model; garis titik-titik = epoch dengan val_loss terendah

Tabel 15. Nilai pada epoch asal bobot yang dievaluasi untuk setiap model

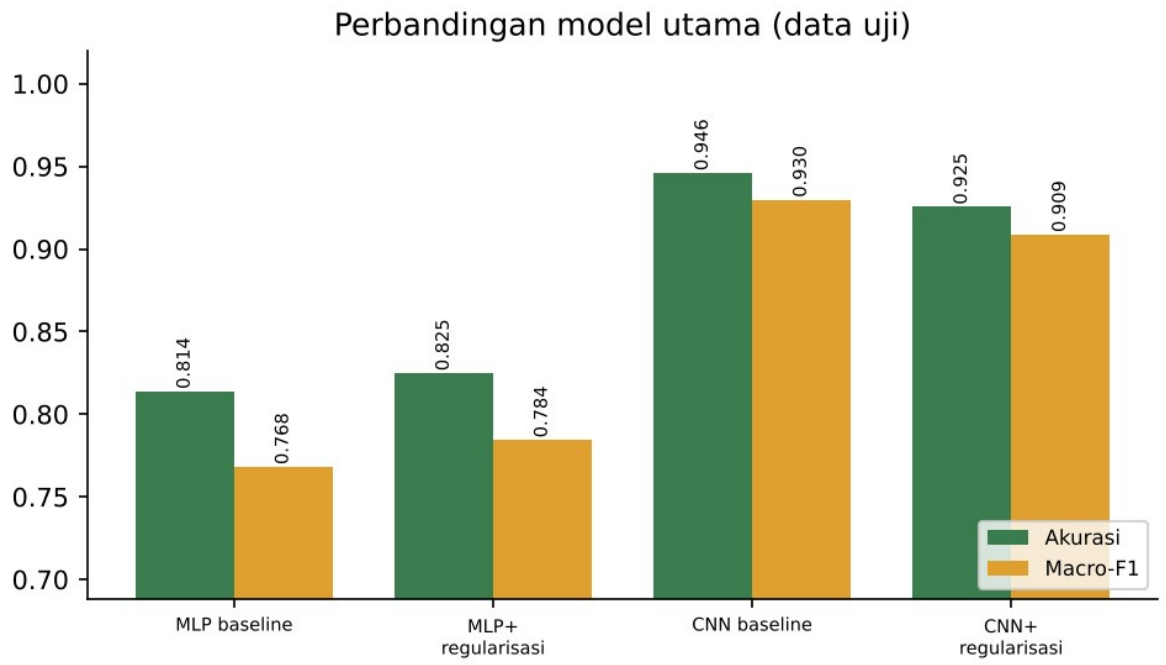
Model	Epoch evaluasi / dijalankan	Train acc	Val acc	Selisih	Train loss	Val loss
MLP baseline	21 / 27	0,8282	0,8178	0,0104	0,4924	0,5394
MLP + regularisasi	18 / 18	0,7543	0,8270	-0,0726	0,8509	0,6526
CNN baseline	15 / 21	0,9768	0,9489	0,0278	0,0701	0,1653
CNN + regularisasi	18 / 18	0,9480	0,9148	0,0332	0,2117	0,2712

Catatan: pada model baseline *early stopping* aktif sehingga bobot terbaik (val_loss terendah) dipulihkan; pada model teregulasi seluruh 18 epoch dijalankan sehingga yang dievaluasi adalah bobot akhir. Metrik *training* pada model teregulasi dihitung pada citra yang sudah diaugmentasi dan dengan Dropout aktif, sehingga cenderung lebih rendah daripada akurasi pada citra latih tanpa augmentasi; selisih train-val pada model teregulasi karenanya tidak sebanding secara langsung dengan model baseline.

A.6.d Perbandingan antarskenario

Tabel 16. Perbandingan seluruh skenario yang dijalankan (data uji; satu seed)

Skenario	Param.	Epoch	Detik/ epoch	Val acc	Test acc	Test macro-F1
MLP baseline	3.180.170	21/27	5.3	0,8178	0,8139	0,7682
MLP + regularisasi	3.181.322	18/18	19.3	0,8270	0,8249	0,7844
CNN baseline	684.490	15/21	32.3	0,9489	0,9460	0,9299
CNN + regularisasi	99.450	14/18	42.6	0,9302	0,9255	0,9088



Gambar 14. Perbandingan accuracy dan macro-F1 antarskenario pada data uji

- Pada versi *baseline*, CNN mencapai macro-F1 0,9299 dan akurasi 94,60% dibandingkan MLP 0,7682 dan 81,39%, selisih macro-F1 0,1617 dengan parameter 4,6 kali lebih sedikit.
- Resep optimasi dan regularisasi meningkatkan MLP: macro-F1 0,7682 → 0,7844 (akurasi 81,39% → 82,49%).
- Pada CNN, resep yang sama tidak meningkatkan kinerja uji: macro-F1 0,9299 → 0,9088 (akurasi 94,60% → 92,55%), dengan jumlah parameter 6,9 kali lebih sedikit (684.490 → 99.450).

Skenario ablasi tambahan (optimizer, tanpa augmentasi, tanpa class weight, tanpa LR scheduling) belum dijalankan pada laporan ini karena keterbatasan waktu komputasi di CPU;

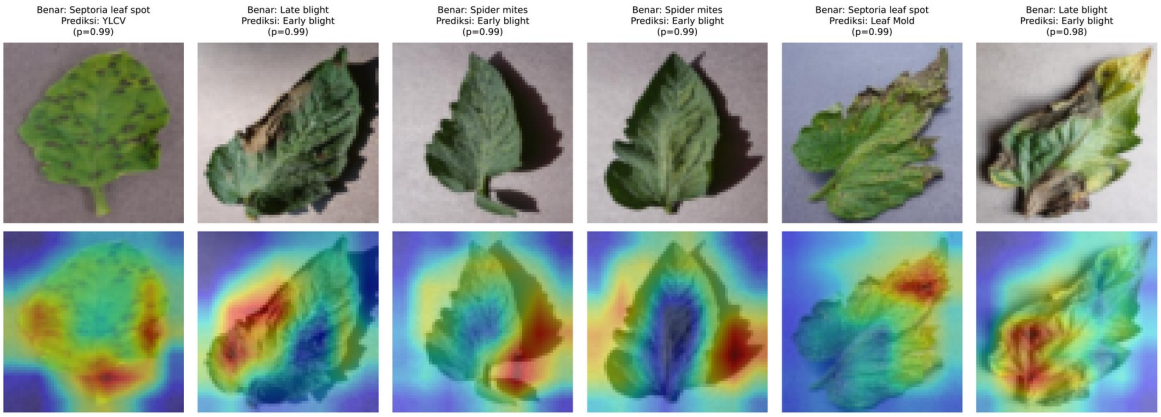
perbandingan antarskenario pada laporan ini terbatas pada empat model utama dan pada perbandingan dengan/tanpa fitur unik (Bab A.5.e).

Karena setiap skenario hanya dijalankan satu kali dengan seed 42, selisih kecil (mendekati satu poin persentase atau kurang) tidak dapat disimpulkan sebagai perbedaan yang pasti; variasi antarseed tidak diukur.

A.6.e Analisis hasil

Kelas yang sering salah. Pada CNN + regularisasi, kelas dengan F1 terendah adalah Early blight (F1 0,761, 150 citra uji), Late blight (0,870), dan Target Spot (0,872). Pasangan kesalahan terbanyak: Late blight diprediksi sebagai Early blight (49 citra); Bacterial spot diprediksi sebagai YLCV (20 citra); Target Spot diprediksi sebagai Spider mites (18 citra); Target Spot diprediksi sebagai Early blight (14 citra).

Kemungkinan penyebab. (1) Kemiripan visual antarpenyakit berbintik atau bercak (bercak bakteri, bercak target, septoria, hawar awal) yang detail halusnya hilang pada resolusi 64x64; (2) kelas minoritas dengan sedikit contoh uji (mis. virus mosaik) membuat metrik per kelas mudah berfluktuasi dengan satu-dua kesalahan; (3) gejala dini atau ringan yang tampak mirip daun sehat; (4) satu seed dan satu kali pembagian data sehingga ada ketidakpastian statistik.



Gambar 15. Grad-CAM pada prediksi yang salah dengan keyakinan tertinggi (Grad-CAM untuk kelas yang diprediksi)

Overfitting/underfitting. CNN baseline mencapai akurasi latih 97,68% dan validasi 94,89% pada epoch 15 (selisih 2,78%), dan val_loss mulai naik setelahnya, tanda *overfitting* ringan. MLP baseline hanya mencapai 82,82% (latih) dan 81,78% (validasi): akurasinya terbatas oleh kemampuan representasi MLP terhadap pola spasial, bukan semata *overfitting*. CNN + regularisasi mencapai 94,80% (latih, dengan augmentasi dan dropout aktif) dan 91,48% (validasi) pada epoch 18 dari 18; selisih latih-validasi hanya 3,32%.

Pemilihan model untuk deployment. Aplikasi memakai CNN + regularisasi walaupun CNN baseline lebih akurat pada data uji (94,60% dibanding 92,55%; selisih 2,06 poin persentase). Alasannya adalah ukuran: 99.450 parameter dan berkas 1.255 KB dibanding 684.490 parameter pada CNN baseline (6,9 kali lebih besar), serta kepala GAP + Dense yang membuat Grad-CAM setara CAM. Ini adalah kompromi yang disengaja antara akurasi dan ukuran; bila akurasi menjadi prioritas utama, CNN baseline atau CNN teregulasi dengan pelatihan lebih panjang lebih tepat dipilih, dan fitur Grad-CAM serta confidence threshold dapat diterapkan pada keduanya.

Stabilitas pelatihan. Pada CNN + regularisasi (iterasi final), akurasi validasi pada delapan epoch pertama berkisar antara 0,55 dan 0,85, dan pada lima epoch terakhir antara 0,907 dan

0,930. Osilasi awal masih cukup lebar, sebanding dengan iterasi sebelumnya (0,18-0,92 dan 0,34-0,92), dan validasi menjadi stabil setelah *learning rate* menurun. Karena L2 dan jadwal *learning rate* diubah bersamaan, tidak dapat dipastikan perubahan mana yang paling berperan; dugaan lain, yaitu perbedaan statistik antara citra teraugmentasi (latih) dan citra asli (validasi) yang memengaruhi Batch Normalization, tidak diuji. Bobot yang dievaluasi adalah bobot akhir pelatihan (atau bobot dengan val_loss terendah bila *early stopping* aktif).

Pengaruh artefak latar hitam. Pada data uji terdapat 26 citra berlatar hitam (seluruhnya Late blight); akurasi model pada subset ini 100,0%, sedangkan recall Late blight pada 259 citra Late blight berlatar normal hanya 74,9%. Citra berlatar hitam lebih mudah dikenali daripada Late blight berlatar normal. Seluruh kesalahan Late blight, termasuk 49 citra yang diprediksi sebagai Early blight (kesalahan terbanyak model), terjadi pada citra berlatar normal. Di sisi lain hanya 0,04% citra non-Late-blight yang diprediksi sebagai Late blight, dan akurasi pada seluruh citra non-hitam 92,48%.

Untuk memeriksa ketergantungan pada latar, Grad-CAM dihitung pada 26 citra uji berlatar hitam: piksel hitam menempati rata-rata 72,1% luas citra dan 73,8% massa Grad-CAM jatuh pada piksel tersebut (rasio 1,02 terhadap luasnya). Massa Grad-CAM pada latar sebanding dengan luas latar sehingga tidak ada bukti perhatian berlebih pada latar; namun karena latar menempati sebagian besar citra, hasil ini tidak dapat membuktikan maupun menyingkirkan jalan pintas lewat latar hitam. Pola recall (100% pada latar hitam, 74,9% pada latar normal) menunjukkan bahwa Late blight berlatar normal lebih sulit, dan dugaan jalan pintas perlu diuji lebih lanjut dengan data tanpa artefak (mis. membuang citra berlatar hitam atau menyamakan latar), yang tidak dilakukan pada proyek ini.

Keterbatasan. (1) Seluruh angka berasal dari satu seed dan satu pembagian data. (2) Masukan hanya 64x64 sehingga detail halus hilang. (3) Dataset PlantVillage berlatar polos dan terkontrol, sehingga performa pada foto lapangan dengan latar kompleks kemungkinan lebih rendah dan tidak diukur di sini; uji non-tomat juga memakai foto dari repositori yang sama (latar serupa), sehingga hasilnya cenderung lebih optimistis daripada kondisi nyata. (4) Eksperimen dijalankan di CPU dengan batas 30 epoch (baseline) dan 18 epoch (teregulasi).

A.7 Teknologi yang Digunakan

Tabel 17. Teknologi, framework, dan tools

Kategori	Teknologi	Versi / spesifikasi
Bahasa pemrograman	Python	3.12.3
Deep learning	TensorFlow (tensorflow-cpu) dan Keras	TensorFlow 2.21.0; Keras 3.15.1
Komputasi numerik	NumPy	2.5.4
Evaluasi model	scikit-learn	1.9.1
Visualisasi	Matplotlib	3.11.2
Pemrosesan citra	Pillow (PIL)	13.0.0 (persiapan data); 12.3.0 (analisis dan aplikasi)
Antarmuka aplikasi	Gradio	6.31.0
Platform hosting	Hugging Face Spaces (SDK Gradio)	CPU basic
Version control	Git dan GitHub	-
Lingkungan pengembangan	Kontainer Linux (Ubuntu) tanpa GPU	1 vCPU, RAM 3 GB, tanpa GPU

Seluruh pelatihan dan evaluasi dijalankan di CPU tanpa GPU, sehingga resolusi masukan dibatasi 64x64 dan model dirancang ringan. Seluruh skrip memakai seed 42; menjalankan ulang di Google Colab atau di GPU dapat menghasilkan angka yang sedikit berbeda karena operasi non-deterministik pada pustaka numerik. **Jika eksperimen dijalankan ulang, seluruh angka pada laporan, video, dan README harus diperbarui agar tetap konsisten.**

A.8 Deployment dan Demo

A.8.a Link deployment

- Aplikasi (Hugging Face Spaces): [tempel tautan Space di sini]
- Repository GitHub (kode, README, notebook, hasil): [tempel tautan GitHub di sini]

A.8.b Tahapan deployment langkah demi langkah

Langkah 1. Menyimpan model. Model akhir (CNN + regularisasi) disimpan dalam format Keras native: `model.save("CNN_reg_final.keras")` (sekitar 1.255 KB). Ambang keyakinan hasil seleksi pada data validasi disimpan di `model/config.json`.

[Tangkapan layar: folder model/ berisi CNN_reg_final.keras dan config.json]

Langkah 2. Menyiapkan skrip inferensi. Berkas `inference.py` berisi kelas `Predictor`: fungsi `preprocess` (resize bilinear 64x64 dan rescale /255, identik dengan pipeline pelatihan), `predict` (probabilitas, kelas, keputusan terima/tolak berdasarkan ambang), dan `gradcam` (peta Grad-CAM).

[Tangkapan layar: isi inference.py di editor]

Langkah 3. Membangun antarmuka. Berkas `app.py` membangun antarmuka Gradio: unggah foto daun, slider ambang keyakinan, tombol klasifikasi, keluaran tiga prediksi teratas, keputusan terima/tolak, dan citra Grad-CAM. Folder `examples/` berisi 10 citra uji (satu per kelas), 2 citra daun non-tomat, dan 1 contoh yang salah diprediksi.

[Tangkapan layar: tampilan antarmuka di komputer lokal (python app.py)]

Langkah 4. Mencatat dependensi. Berkas `requirements.txt` mencatat `tensorflow-cpu==2.21.0`, `numpy`, `pillow`, `matplotlib`, dan `gradio`. Berkas `README.md` pada folder aplikasi berisi konfigurasi Space (SDK `gradio`, `app_file: app.py`).

[Tangkapan layar: isi requirements.txt dan README.md]

Langkah 5. Mengunggah ke platform hosting. Buat Space baru di Hugging Face (SDK: Gradio, hardware: CPU basic), lalu unggah seluruh isi folder `app/` (melalui antarmuka web *Files* > *Add file* atau `git push`). Space membangun lingkungan secara otomatis dari `requirements.txt`.

[Tangkapan layar: halaman pembuatan Space dan log build berstatus Running]

Langkah 6. Menguji aplikasi. Uji dengan contoh dari folder `examples/`: satu prediksi benar, satu contoh daun non-tomat (harus ditolak), dan variasi slider ambang. Pastikan hasil sama dengan yang dilaporkan pada Bab A.6.

[Tangkapan layar: hasil prediksi benar, prediksi ditolak, dan peta Grad-CAM pada Space]

A.8.c Penjelasan video demo dan deployment

Video demo berdurasi sekitar 8-10 menit dan direkam oleh seluruh anggota kelompok secara bergantian. Isi video mengikuti urutan berikut (naskah lengkap tersedia pada berkas `docs/Naskah_Video_Demo.md`):

Tabel 18. Rencana isi video demo

No	Bagian	Isi yang ditampilkan	Penyaji
1	Pembukaan	Judul, tujuan, dataset, dan ringkasan hasil (MLP vs CNN)	[nama]

No	Bagian	Isi yang ditampilkan	Penyaji
2	Tampilan antarmuka	Halaman Space: area unggah, slider ambang, tombol, panel keluaran	[nama]
3	Contoh input dan output	Satu prediksi benar dan satu prediksi salah dari data uji (disertai penjelasan penyebabnya)	[nama]
4	Fitur unik 1: Grad-CAM	Peta Grad-CAM pada prediksi benar dan salah; hasil uji deletion	[nama]
5	Fitur unik 2: confidence threshold	Gambar daun non-tomat ditolak; geser slider ambang; kurva selective classification	[nama]
6	Tahapan deployment	Simpan model, inferensi, antarmuka, dependensi, unggah ke Spaces, uji	[nama]
7	Penutup	Ringkasan hasil, keterbatasan, dan pengembangan lanjutan	[nama]

Contoh masukan dan keluaran yang diperagakan pada video adalah seluruh berkas pada folder `app/examples`; hasil berikut diperoleh dari model akhir dengan ambang tau = 0,945. Contoh ini sengaja memuat prediksi benar, prediksi salah, dan kegagalan fitur agar keterbatasan model terlihat jujur.

Tabel 19. Hasil model akhir pada contoh aplikasi (tau = 0,945)

Berkas	Label sebenarnya	Prediksi	Keyakinan	Keputusan	Keterangan
00_Bacterial_spot	Bacterial spot	Bacterial spot	99,4%	Diterima	Benar dan diterima
01_Early_blight	Early blight	Bacterial spot	74,2%	Ditolak	Salah dan ditolak: threshold mencegah diagnosis keliru
02_Late_blight	Late blight	Late blight	99,5%	Diterima	Benar dan diterima
03_Leaf_Mold	Leaf Mold	Leaf Mold	99,9%	Diterima	Benar dan diterima
04_Septoria_leaf_spot	Septoria leaf spot	Septoria leaf spot	96,9%	Diterima	Benar dan diterima
05_Spider_mites	Spider mites	Spider mites	87,7%	Ditolak	Benar tetapi ditolak (keyakinan < tau): contoh kompromi threshold
06_Target_Spot	Target Spot	Target Spot	37,0%	Ditolak	Benar tetapi ditolak (keyakinan < tau): contoh kompromi threshold
07_YLCV	YLCV	YLCV	100,0%	Diterima	Benar dan diterima
08_Mosaic_virus	Mosaic virus	Mosaic virus	98,8%	Diterima	Benar dan diterima
09_healthy	healthy	healthy	100,0%	Diterima	Benar dan diterima
ood_Pepper	non-tomat (Pepper)	Septoria leaf spot	98,6%	Diterima	Non-tomat lolos (kegagalan deteksi di luar domain)
ood_Potato	non-tomat (Potato)	Target Spot	69,1%	Ditolak	Non-tomat ditolak
salah_Septoria_jadi_YLCV	Septoria leaf spot	YLCV	99,1%	Diterima	Salah dengan keyakinan tinggi dan diterima: keterbatasan threshold

Angka yang disebut dalam video harus sama dengan Bab A.6 (laporan) dan keluaran skrip pada repository (konsistensi antarartefak).

A.9 Lampiran

A.9.a Cuplikan kode penting

Cuplikan berikut diambil langsung dari berkas pada repository sehingga identik dengan kode yang menghasilkan angka pada laporan ini.

Kode 1. Preprocessing: muat citra, hash near-duplicate, dan pembagian data (`prep_data.py`)

```
def load(path):
    im = Image.open(path).convert("RGB").resize((IMG, IMG), Image.BILINEAR)
    return np.asarray(im, dtype=np.uint8)


def dhash(arr):
    """dHash 256-bit (16 baris x 16 perbandingan) -> 4 kata uint64."""
    g = np.asarray(Image.fromarray(arr).convert("L").resize((17, 16), Image.BILINEAR),
dtype=np.int16)
    bits = (g[:, 1:] > g[:, :-1]).flatten()
    return np.packbits(bits).view(np.uint64)

# --- near-duplicate (Hamming <= HAM_THR dari 256 bit dHash) ---
HAM_THR = 12
N = len(X)
parent = np.arange(N)


def find(i):
    while parent[i] != i:
        parent[i] = parent[parent[i]]
        i = parent[i]
    return i


pairs = 0
for i in range(0, N, 512):
    blk = hashes[i:i + 512]
    xor = blk[:, None, :] ^ hashes[None, :, :]
    ham = np.bitwise_count(xor).sum(axis=2)
    ii, jj = np.where(ham <= HAM_THR)
    for a, b in zip(ii + i, jj):
        if a < b:
            ra, rb = find(a), find(b)
            if ra != rb:
                parent[ra] = rb
            pairs += 1
groups = np.array([find(i) for i in range(N)])
_, groups = np.unique(groups, return_inverse=True)
sizes = np.bincount(groups)
n_groups_multi = int((sizes > 1).sum())
cross_class = 0
for g in np.where(sizes > 1)[0]:
    if len(set(y[groups == g])) > 1:
        cross_class += 1
print(f"Pasangan near-duplicate: {pairs} | grup ukuran>1: {n_groups_multi} | "
      f"citra dalam grup tsb: {int(sizes[sizes>1].sum())} | grup lintas-kelas: {cross_class}")

# --- split 70/15/15 stratified + group ---
sgkf = StratifiedGroupKFold(n_splits=20, shuffle=True, random_state=SEED)  # tiap fold ~5%
folds = [te for _, te in sgkf.split(X, y, groups)]
test_idx = np.concatenate(folds[:3])        # 15%
val_idx = np.concatenate(folds[3:6])        # 15%
```

```
train_idx = np.concatenate(folds[6:])       # 70%
assert len(set(groups[train_idx]) & set(groups[val_idx])) == 0
assert len(set(groups[train_idx]) & set(groups[test_idx])) == 0
assert len(set(groups[val_idx]) & set(groups[test_idx])) == 0
print("Split:", len(train_idx), len(val_idx), len(test_idx))
for nm, idx in [("train", train_idx), ("val", val_idx), ("test", test_idx)]:
    print(nm, np.bincount(y[idx], minlength=10).tolist())
```

Kode 2. Input pipeline tf.data, class weight, dan augmentasi (`common.py`)

```
def make_ds(X, y, training, batch=BATCH, seed=SEED):
    """Input pipeline (materi P1-P2): from_tensor_slices -> shuffle -> map(scale) -> batch ->
prefetch."""
    ds = tf.data.Dataset.from_tensor_slices((X, y))
    if training:
        ds = ds.shuffle(len(X), seed=seed, reshuffle_each_iteration=True)
    ds = ds.map(lambda a, b: (tf.cast(a, tf.float32) / 255.0, b),
num_parallel_calls=tf.data.AUTOTUNE)
    return ds.batch(batch).prefetch(tf.data.AUTOTUNE)


def class_weights(y):
    cnt = np.bincount(y, minlength=NC)
    return {i: float(len(y) / (NC * cnt[i])) for i in range(NC)}


def augmentation():
    """Augmentasi citra (materi P3); aktif hanya saat training."""
    return keras.Sequential([
        layers.RandomFlip("horizontal_and_vertical"),
        layers.RandomRotation(0.08),
        layers.RandomZoom(0.1),
        layers.RandomBrightness(0.15, value_range=(0.0, 1.0)),
    ], name="augmentasi")
```

Kode 3a. Arsitektur MLP + regularisasi (`common.py`)

```
def build_mlp_reg(l2=1e-4, drop=0.3, aug=True):
    reg = regularizers.l2(l2)
    m = [layers.Input(shape=(IMG, IMG, 3))]
    if aug:
        m.append(augmentation())
    m.append(layers.Flatten())
    for u in (256, 128):                        # Dense -> BN -> ReLU -> Dropout (materi P4)
        m += [layers.Dense(u, use_bias=False, kernel_regularizer=reg),
              layers.BatchNormalization(momentum=BN_MOMENTUM), layers.Activation("relu"),
layers.Dropout(drop)]
    m.append(layers.Dense(NC, activation="softmax"))
    return keras.Sequential(m, name="MLP_reg")
```

Kode 3b. Arsitektur CNN baseline dan CNN + regularisasi (`common.py`)

```
def build_cnn_baseline():
    """CNN dasar sesuai materi P6: Conv-Pool x3 -> Flatten -> Dense."""
    return keras.Sequential([
        layers.Input(shape=(IMG, IMG, 3)),
        layers.Conv2D(32, 3, activation="relu"), layers.MaxPooling2D(),
        layers.Conv2D(64, 3, activation="relu"), layers.MaxPooling2D(),
        layers.Conv2D(128, 3, activation="relu"), layers.MaxPooling2D(),
        layers.Flatten(),
        layers.Dense(128, activation="relu"),
        layers.Dense(NC, activation="softmax"),
```

```
    ], name="CNN_baseline")


def build_cnn_reg(drop=0.3, l2=1e-4, aug=True, filters=(16, 32, 64, 128)):
    """CNN teregulasi (model akhir). Functional API agar Grad-CAM dapat mengakses feature map
terakhir."""
    reg = regularizers.l2(l2)
    inp = keras.Input(shape=(IMG, IMG, 3))
    x = augmentation()(inp) if aug else inp
    for i, f in enumerate(filters):
        x = layers.Conv2D(f, 3, padding="same", use_bias=False, kernel_regularizer=reg,
                          name=f"conv{i+1}")(x)
        x = layers.BatchNormalization(momentum=BN_MOMENTUM, name=f"bn{i+1}")(x)
        x = layers.Activation("relu", name=f"relu{i+1}")(x)
        if i < len(filters) - 1:
            x = layers.MaxPooling2D(name=f"pool{i+1}")(x)
    x = layers.GlobalAveragePooling2D(name="gap")(x)
    x = layers.Dropout(drop, name="dropout")(x)
    out = layers.Dense(NC, activation="softmax", kernel_regularizer=reg, name="logits")(x)
    return keras.Model(inp, out, name="CNN_reg")
```

Kode 4. Fungsi pelatihan dengan callback (`common.py`)

```
def train(model, name, data, optimizer, epochs=30, use_cw=True, sched=None, es_patience=6,
schedule_note=""):
    """sched: None (lr konstan) | "plateau" (ReduceLROnPlateau) | ("cosine", lr0) (cosine
annealing selama `epochs`).
    Pelatihan dapat dilanjutkan otomatis dari epoch terakhir (BackupAndRestore) jika proses
terputus."""
    (Xtr, ytr), (Xva, yva), (Xte, yte) = data[:3]
    keras.utils.set_random_seed(SEED)
    train_ds, val_ds, test_ds = make_ds(Xtr, ytr, True), make_ds(Xva, yva, False), make_ds(Xte,
yte, False)
    model.compile(optimizer=optimizer, loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
    hl = HistLogger(f"{RES}/{name}_history_partial.json")
    cbs = [keras.callbacks.EarlyStopping(monitor="val_loss", patience=es_patience,
restore_best_weights=True), hl,
           keras.callbacks.BackupAndRestore(f"{RES}/backup_{name}")]
    if sched == "plateau":
        cbs.append(keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3,
min_lr=1e-5))
    elif isinstance(sched, tuple) and sched[0] == "cosine":
        cbs.append(keras.callbacks.LearningRateScheduler(cosine_lr(sched[1], epochs)))
    hist = model.fit(train_ds, validation_data=val_ds, epochs=epochs,
                     class_weight=class_weights(ytr) if use_cw else None, callbacks=cbs,
verbose=2)
    h = hl.h
    res = evaluate(model, test_ds, yte)
    res_val = evaluate(model, val_ds, yva)
    res.update(name=name, params=int(model.count_params()), epochs_run=len(h["loss"]),
               best_epoch=int(np.argmin(h["val_loss"]) + 1),
train_time_s=round(float(sum(h["epoch_time"])), 1),
               sec_per_epoch=round(float(np.mean(h["epoch_time"])), 1), val_acc=res_val["acc"],
               val_macro_f1=res_val["macro_f1"], schedule_note=schedule_note, history=h)
    proba = res.pop("_proba")
    json.dump(res, open(f"{RES}/{name}.json", "w"))
    np.save(f"{RES}/{name}_test_proba.npy", proba)
    return model, res
```

Kode 5a. Fitur unik 1 dan 2: kelas Predictor dengan Grad-CAM dan keputusan threshold (`inference.py`)

```
class Predictor:
    def __init__(self, model_path, tau=0.90):
        self.model = keras.models.load_model(model_path, compile=False)
        self.tau = tau
        self._cam_model = keras.Model(self.model.inputs,
[self.model.get_layer(LAST_CONV_ACT).output])
        self._W, self._b = self.model.get_layer("logits").get_weights()

    def proba(self, x):
        return self.model.predict(x, verbose=0)

    def gradcam(self, x, class_idx):
        """Grad-CAM (Selvaraju dkk., 2017) terhadap logit kelas `class_idx` pada feature map
konvolusi terakhir."""
        x = tf.convert_to_tensor(x)
        with tf.GradientTape() as tape:
            fmap = self._cam_model(x, training=False)
            tape.watch(fmap)
            gap = tf.reduce_mean(fmap, axis=(1, 2))
            logits = tf.matmul(gap, self._W) + self._b
            score = logits[:, class_idx]
        grads = tape.gradient(score, fmap)
        alpha = tf.reduce_mean(grads, axis=(1, 2), keepdims=True)
        cam = tf.nn.relu(tf.reduce_sum(alpha * fmap, axis=-1))[0]
        cam = cam / (tf.reduce_max(cam) + 1e-8)
        cam = tf.image.resize(cam[..., None], (IMG, IMG), method="bilinear")[..., 0]
        return cam.numpy()

    def predict(self, img, with_cam=True):
        x = preprocess(img)
        p = self.proba(x)[0]
        k = int(p.argmax())
        out = dict(proba=p, index=k, label=CLASS_NAMES[k], confidence=float(p[k]),
                   accepted=bool(p[k] >= self.tau), x=x[0])
        if with_cam:
            out["cam"] = self.gradcam(x, k)
        return out
```

Kode 5b. Pemilihan ambang tau pada data validasi dan uji OOD (`analysis.py`)

```
def sel_stats(m, c, t):
    a = m >= t; return float(a.mean()), (float(c[a].mean()) if a.any() else float("nan"))
grid = np.round(np.arange(0.30, 0.9999, 0.005), 3)
TARGET = 0.99
tau = next((t for t in grid if sel_stats(mva, cva, t)[1] >= TARGET), 0.9)
OUT["tau"] = float(tau); OUT["tau_rule"] = f"nilai terkecil pada grid 0.005 sehingga akurasi
prediksi-diterima pada validation >= {TARGET}"
```

Kode 5c. Uji deletion untuk Grad-CAM (`analysis.py`)

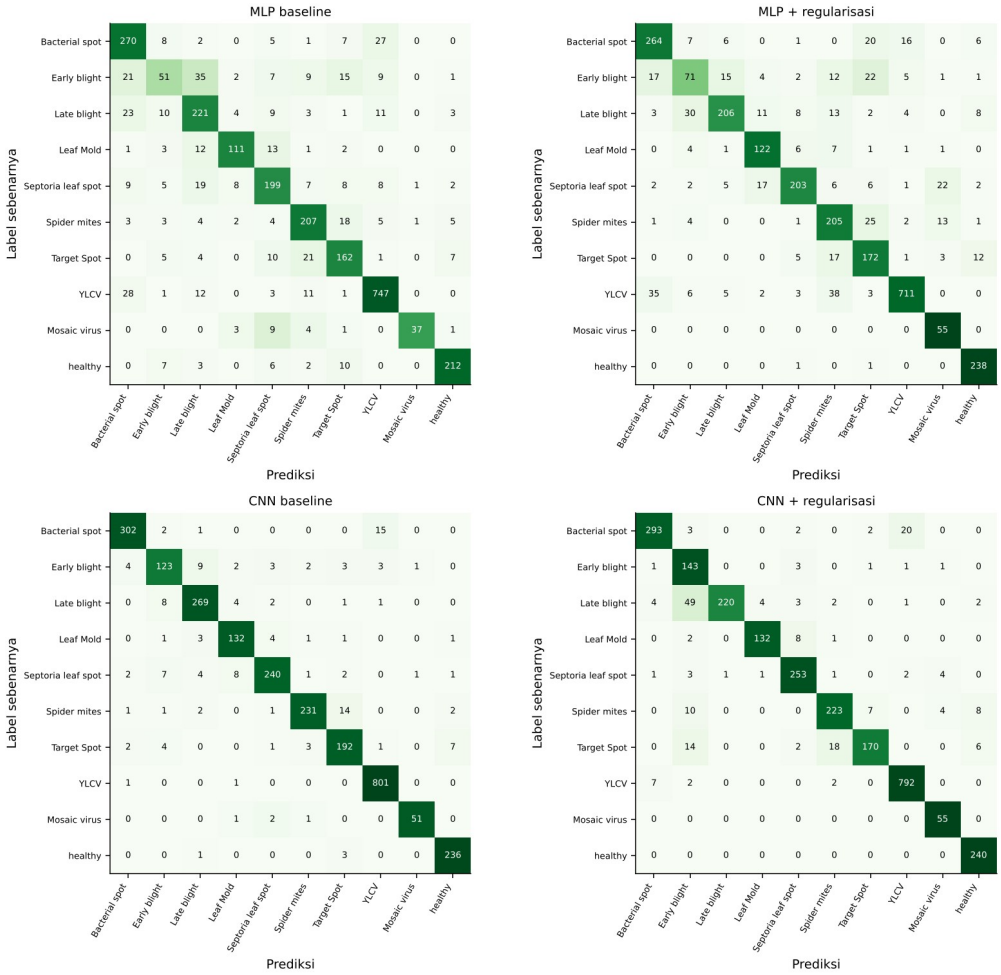
```
def mask_eval(order_fn):
    out = []
    for p in fr:
        xs = Xte_f[samp].copy(); kpx = int(p * HW)
        for i in range(len(samp)):
            if kpx == 0: continue
            sel_px = order_fn(i)[:kpx]; flat = xs[i].reshape(-1, 3); flat[sel_px] =
Xte_f[samp[i]].mean(axis=(0, 1)); xs[i] = flat.reshape(IMG, IMG, 3)
```

```
        pb = pr.proba(xs); out.append((pb[np.arange(len(samp)), yte[samp]].mean(), (pb.argmax(1)
== yte[samp]).mean()))
    return np.array(out)
tie = rng.random((len(samp), HW)) * 1e-6
cam_order = lambda i: np.argsort(-(cams[i].flatten() + tie[i]))
D_cam = mask_eval(cam_order)
D_rand = np.mean([mask_eval(lambda i, r=np.random.default_rng(s): r.permutation(HW)) for s in
range(3)], axis=0)
inv_order = lambda i: np.argsort((cams[i].flatten() + tie[i]))  # piksel PALING TIDAK penting
dihapus lebih dulu
D_inv = mask_eval(inv_order)
# kontrol kontigu: gunakan peta Grad-CAM dari citra LAIN (tanpa titik tetap) sebagai urutan
penghapusan
_sh = rng.permutation(len(samp)); perm = np.empty(len(samp), dtype=int); perm[_sh] = np.roll(_sh,
1)
perm_order = lambda i: np.argsort(-(cams[perm[i]].flatten() + tie[i]))
D_perm = mask_eval(perm_order)
```

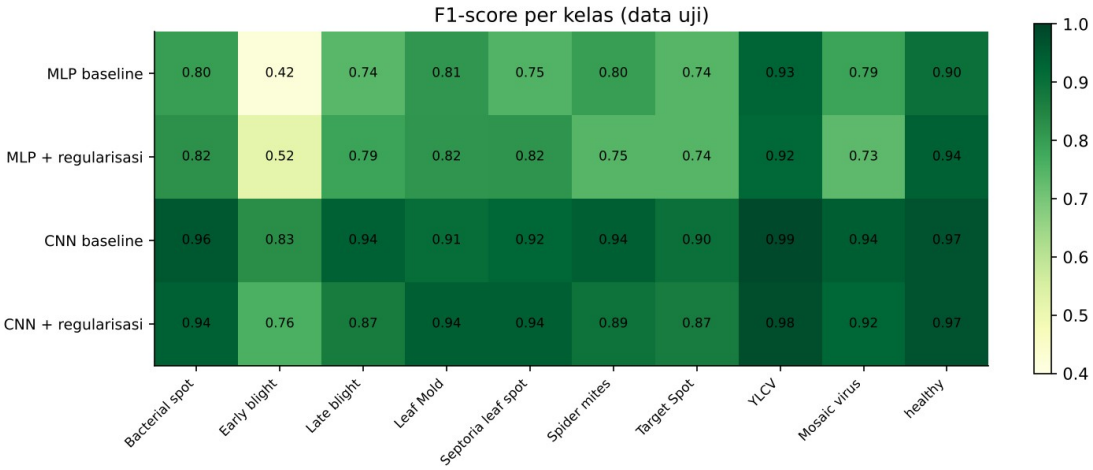
Kode 6. Inferensi pada aplikasi Gradio (`app/app.py`)

```
def run(img, tau):
    if img is None:
        return {}, "Silakan unggah foto satu daun tomat.", None
    predictor.tau = float(tau)
    out = predictor.predict(img, with_cam=True)
    p = out["proba"]
    top = np.argsort(-p)[:3]
    label_dict = {LABEL_ID[CLASS_NAMES[i]]: float(p[i]) for i in top}
    best = LABEL_ID[out["label"]]
    if out["accepted"]:
        verdict = (f"### ✅ {best}\nKeyakinan **{out['confidence']*100:.1f}%** (≥ ambang
{tau:.2f}). "
                   f"Area merah pada peta Grad-CAM menunjukkan bagian daun yang paling
memengaruhi keputusan model.")
    else:
        verdict = (f"### ⚠️ Prediksi ditolak\nKeyakinan tertinggi hanya
**{out['confidence']*100:.1f}%** (< ambang {tau:.2f}) "
                   f"untuk *{best}*. Gambar kemungkinan **bukan daun tomat**, buram, atau berada
di luar kondisi data latih. "
                   f"Coba foto ulang satu daun dengan latar polos dan pencahayaan merata.")
    cam_img = overlay(out["x"], out["cam"])
    cam_img = np.asarray(Image.fromarray((cam_img * 255).astype(np.uint8)).resize((320, 320),
Image.BICUBIC))
    return label_dict, verdict, cam_img
```

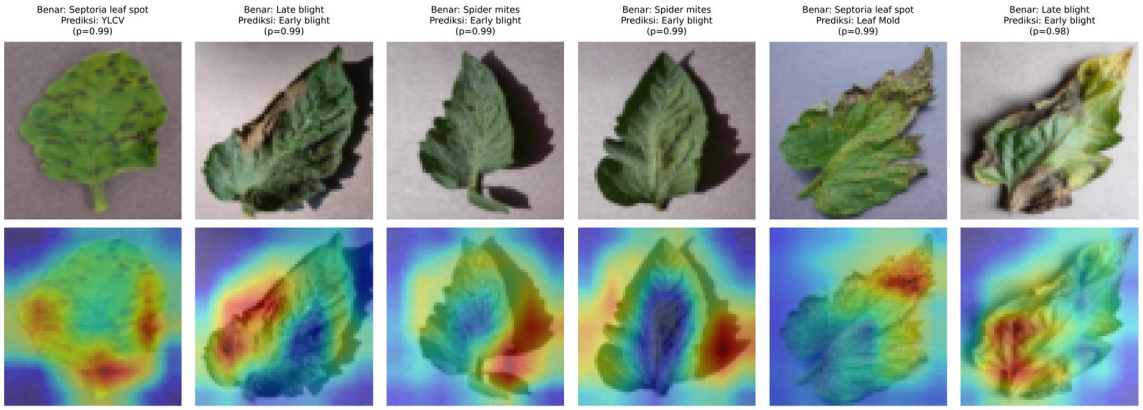
A.9.b Grafik evaluasi



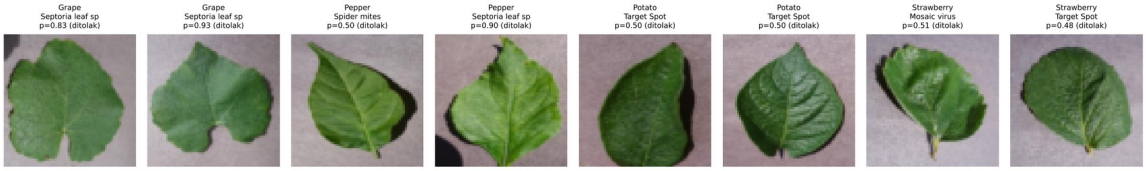
Gambar 16. Confusion matrix keempat model pada data uji



Gambar 17. F1-score per kelas untuk keempat model (data uji)



Gambar 18. Grad-CAM pada contoh prediksi yang salah (kelas prediksi)



Gambar 19. Contoh daun non-tomat beserta keputusan terima/tolak

A.10 Daftar Pustaka

[1] Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. *arXiv:1511.08060*.

[2] Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, *7*, 1419.

[3] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 448-456.

[4] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, *15*(56), 1929-1958.

[5] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

[6] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 618-626.

[7] Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. *International Conference on Learning Representations (ICLR)*.

[8] Geifman, Y., & El-Yaniv, R. (2017). Selective classification for deep neural networks. *Advances in Neural Information Processing Systems (NeurIPS) 30*.

[9] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 265-283.

[10] Chollet, F., et al. (2015). *Keras*. https://keras.io

[11] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825-2830.

[12] Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A., & Zou, J. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. *arXiv:1906.02569*.

[13] Materi kuliah Pembelajaran Mendalam, Pertemuan 1-7 (Pengenalan Deep Learning, Tensor dan Input Pipeline, Perceptron dan MLP, Deep Neural Network dan Optimasi, Convolutional Neural Network).

BAGIAN B. TAUTAN

Pastikan seluruh tautan dapat dibuka tanpa meminta akses.

Tabel 20. Tautan artefak proyek

No	Artefak	Tautan
1	Source code (GitHub)	[tempel tautan GitHub]
2	Video demo beserta tahapan deployment (YouTube / Google Drive)	[tempel tautan video]
3	Aplikasi yang sudah di-deploy (Hugging Face Spaces)	[tempel tautan Space]